HACKERS HRD · 특강

AI X n8n 업무자동화 실전 프로젝트

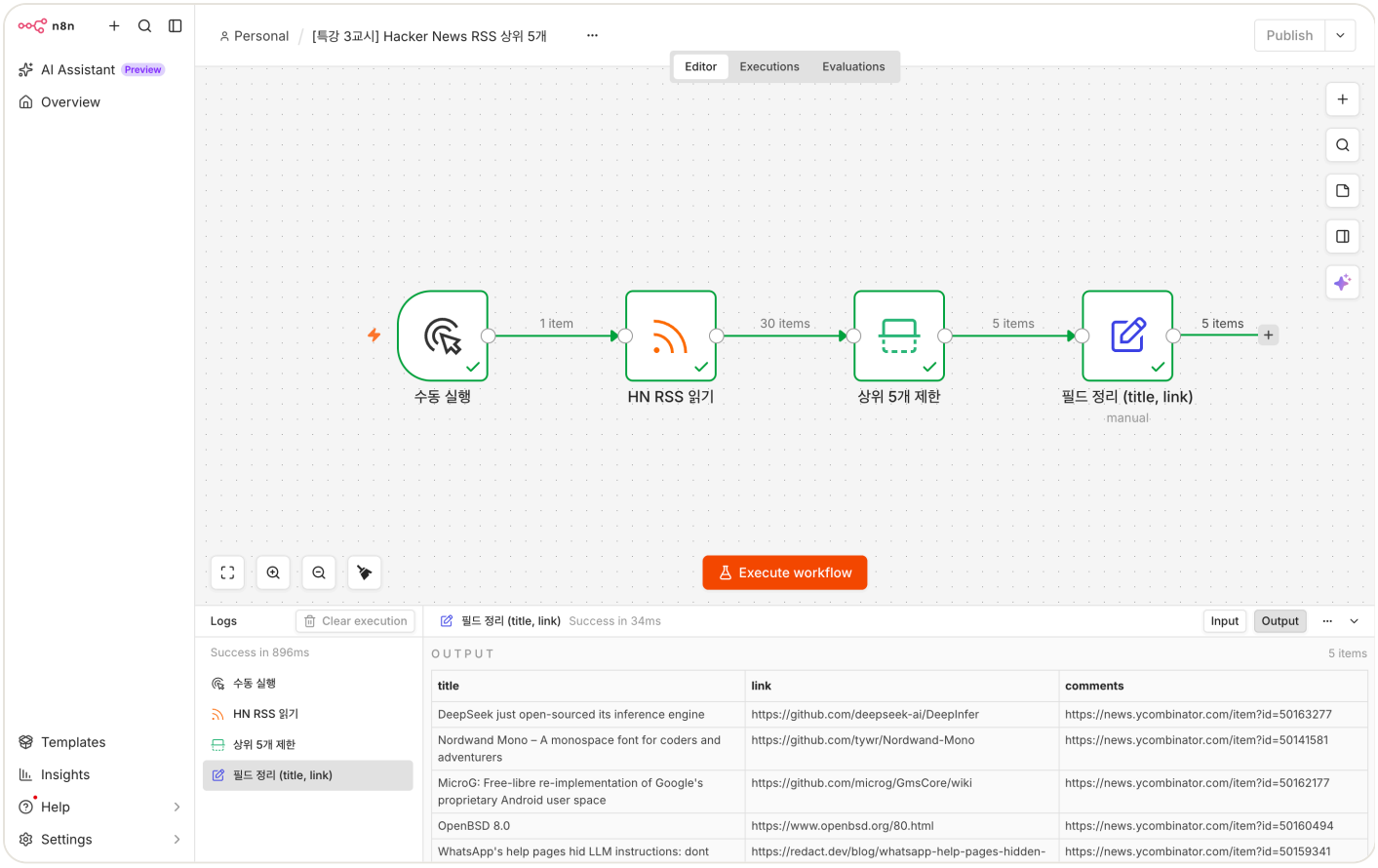
3시간 특강 · 중소조선연구원 · 2026.10.26 (월) 13:00–16:00

오프닝 · n8n 과 AI, 그리고 워크플로우

n8n 이란?

> **n8n = 업무를 “순서도”로 그려두면, 그 순서대로 24시간 대신 일해 주는 자동화 도구**

- 이름 뜻: **n**odemation → n + 8글자 + n (노드로 하는 자동화)
- 화면에서 **블록(노드)을 끌어다 놓고 선으로 연결**하면 끝 — 코딩을 몰라도 시작할 수 있습니다.
- **수백 가지 서비스와 연결**: 메일 · 구글시트 · 슬랙/텔레그램 · 노션 · DB · 웹사이트 · **AI(ChatGPT, Claude, Gemini)** …
- **내 컴퓨터/회사 서버에 설치 가능**(셀프호스팅) → 사내 데이터를 밖으로 보내지 않고 자동화
- 비슷한 도구: Zapier, Make — n8n 은 **자유도 · 가격 · AI 기능**이 강점



n8n 화면 — 블록(노드)을 선으로 이어 만든 워크플로우

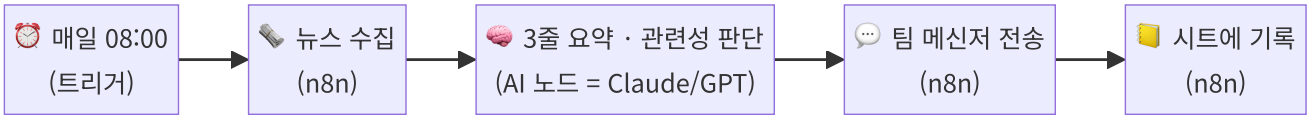
Claude · Codex 와 무엇이 다른가요?

요즘 Claude, ChatGPT, Codex 같은 AI 가 “알아서 다 해준다”는데, n8n 은 왜 필요할까요?

	🧠 Claude · ChatGPT	👩‍💻 Claude Code · Codex	🏭 n8n
한마디로	**똑똑한 상담 직원**	**AI 개발자**	**공장 생산 라인 / 배관**
일하는 방식	내가 물어보면 대답	내가 시키면 코드를 짜고 고침	**정해진 순서**대로 반복 실행
언제 일하나	내가 말을 걸 때만	내가 시킬 때만	**조건이 되면 스스로** (매일 9시, 메일 도착, 버튼 클릭 …)
결과	매번 조금씩 다를 수 있음	매번 다를 수 있음	**항상 같은 순서 · 같은 규칙**
기록	대화창	코드 저장소	**실행마다 단계별 기록** (Executions)
잘하는 일	생각 · 요약 · 글쓰기 · 판단	프로그램 · 웹페이지 만들기	**연결 · 반복 · 정시 실행 · 전달**

💡 경쟁 관계가 아니라 팀입니다.

- n8n 라인 중간에 **AI 노드(Claude/GPT)를 "두뇌 블록"으로 끼워** 요약·분류·판단을 맡기고,
- **Claude Code · Codex · Cursor 에게** 웹페이지나 n8n 워크플로우 초안을 **만들게** 하고,
- 다 만든 n8n 워크플로우는 반대로 **AI 가 부르는 도구(MCP)** 가 되기도 합니다.



정리: AI 는 **"생각"**, n8n 은 **"언제 · 어떤 순서로 · 어디에 연결할지"** 를 맡습니다.

워크플로우를 만든다는 것

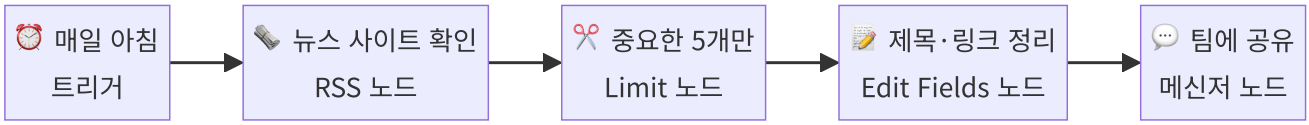
> **워크플로우 = 사람이 하던 일을 "단계"로 쪼개서, 블록으로 이어 붙인 순서도**

요리 레시피와 똑같습니다. 🍳 **"언제(트리거) → 무엇을(노드) → 어떤 순서로(연결선)"**

레시피	워크플로우	n8n 에서는
"저녁 6시가 되면"	**트리거** — 언제 시작?	Schedule · Webhook · 수동 실행
"재료 손질 → 볶기 → 담기"	**노드** — 각 단계의 작업	RSS 읽기 · AI 요약 · 메일 보내기 …
화살표 순서	**연결선** — 결과가 다음 단계로 흘러감	노드 사이의 선
완성된 요리	**결과** — 어디로 보낼까?	메신저 · 시트 · 메일 · 웹페이지

연습: 내 업무를 쪼개 보기

"매일 아침 업계 뉴스를 확인해서 중요한 것만 팀에 공유한다" 👉



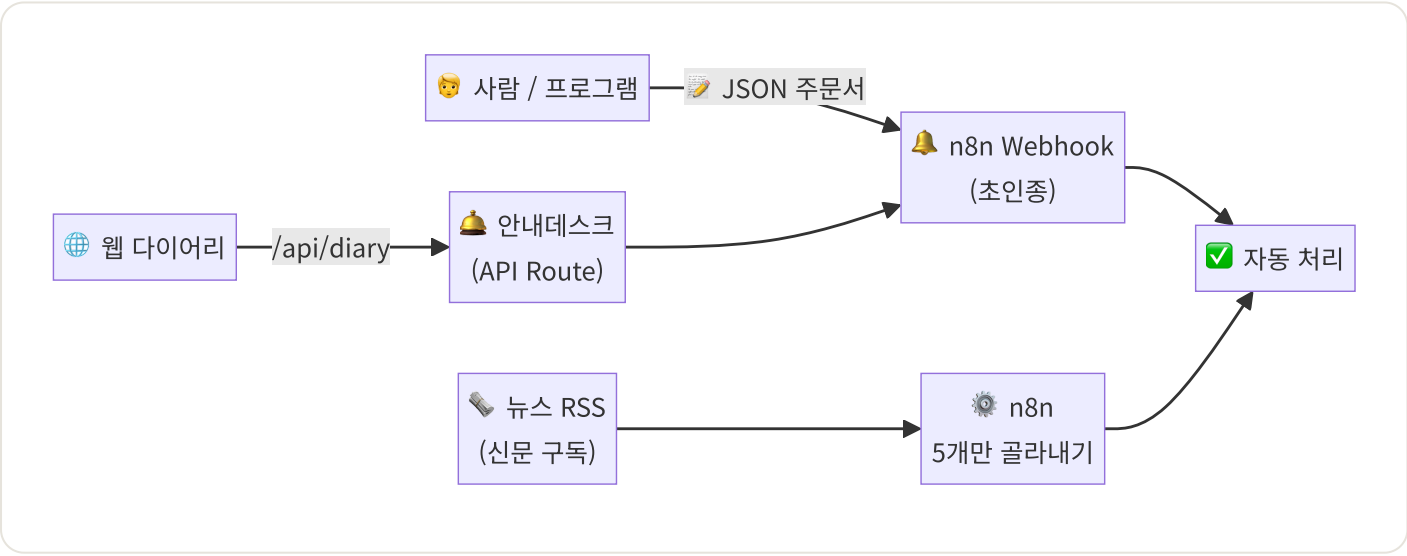
- 사람이 머릿속으로 하던 일을 **눈에 보이는 순서도**로 꺼내는 것 → 이것이 “워크플로우를 만든다”는 뜻
- 한 번 만들면 **매일, 정확하게, 대신** 실행됩니다. 오늘 3교시에 바로 이것(앞의 4단계)을 만듭니다.

오늘의 목표

> **코드를 거의 치지 않고**, 업무 자동화의 “배관” 3개를 끝까지 연결해 봅니다.

자동화는 결국 이 한 문장입니다.

누가 → 무엇을 → 어디로 보내는가



시간	교시	오늘 만들 것	과제
13:00	오프닝	n8n 이란? · AI 와의 차이 · 워크플로우란?	
13:20	**1교시**	n8n이 JSON을 받는 “초인종”	문제 1 (30점)
14:10	**2교시**	웹 다이어리 → 안내데스크 → n8n, 인터넷에 배포	문제 2 (40점)
15:05	**3교시**	최신 뉴스 5개만 골라오기	문제 3 (30점)
15:40	**최신 방법론**	Claude Code 로 n8n 설치부터 미션 해결까지 · Aside Browser	
15:50	마무리 · Q&A	과제 제출 · 질문	

오늘 꼭 가져갈 단어 8개

단어	한 줄 비유
API	🍽️ 식당 **웨이터** — 메뉴판(규칙)대로 주문을 주방에 전달
JSON	📝 정해진 칸이 있는 **주문서**
Webhook	🔔 n8n에 달린 **초인종**
API Route	🛎️ 회사 **안내데스크**
환경변수	🔐 비밀값을 넣어두는 **금고**
배포	🤖 내 식당을 **번화가에 개업**
RSS	🗞️ 신문 **정기구독**
아이템(item)	📦 **컨베이어 벨트 위 상자**

준비물 체크

- [ ] **n8n** 로그인 (n8n Cloud 체험판 또는 설치형)
- [ ] hackers-rest-client.vercel.app (강의 전용 · 설치 없음) 또는 hoppscotch.io 접속 (설치 없음 · 구글 로그인하면 요청 저장) — Cursor + REST Client 도 OK
- [ ] **GitHub** 계정 · **Vercel** 계정 (Continue with GitHub)

> 🙋 막히면 **손 들기** · 스트리밍은 **채팅에** 🙋

📦 **셀프 학습 키트 · 집에서 다시 하기**

- 🎁 **키트 첫 화면** — 매뉴얼 · 워크플로우 · 그림을 한곳에
- 📘 **셀프 실습 매뉴얼** 1장 — n8n Cloud 가입 / Docker 로 내 PC 설치 · Cursor · REST Client

1교시 · n8n Webhook과 REST Client

📚 이번 교시 필수 개념

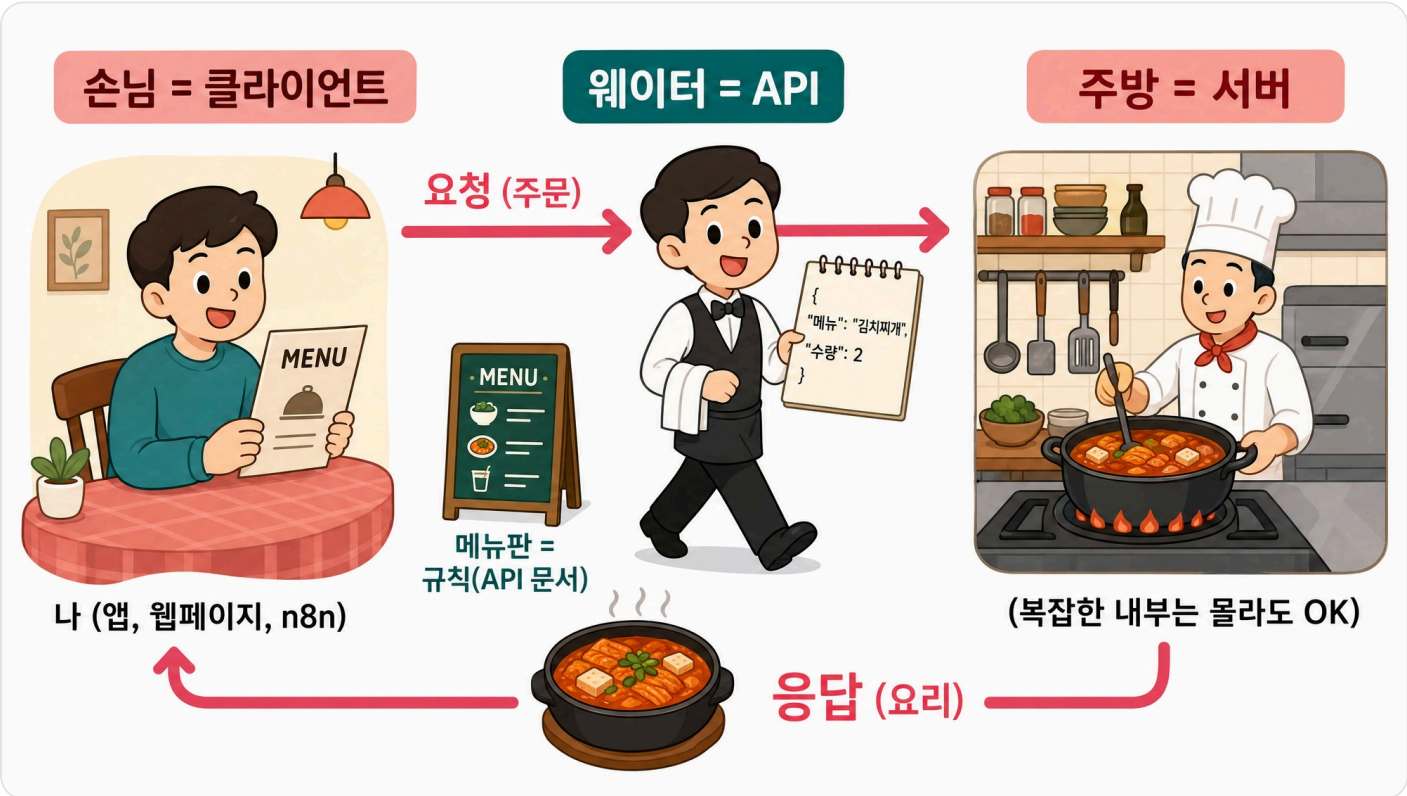
🍽️ API · 📮 URL(주소) · ✉️ GET · POST · 🧾 상태코드 · 📝 JSON

🔔 Webhook · 🎭 Test · Production URL · ⚡ REST Client Web

📨 Hoppscotch · 📁 워크스페이스 · 컬렉션 · 🔀 CORS · Proxy

목표: **"다른 프로그램이 n8n 에게 JSON 주문서를 보내면, n8n 이 받는다"** 를 눈으로 확인

API — 식당 웨이터



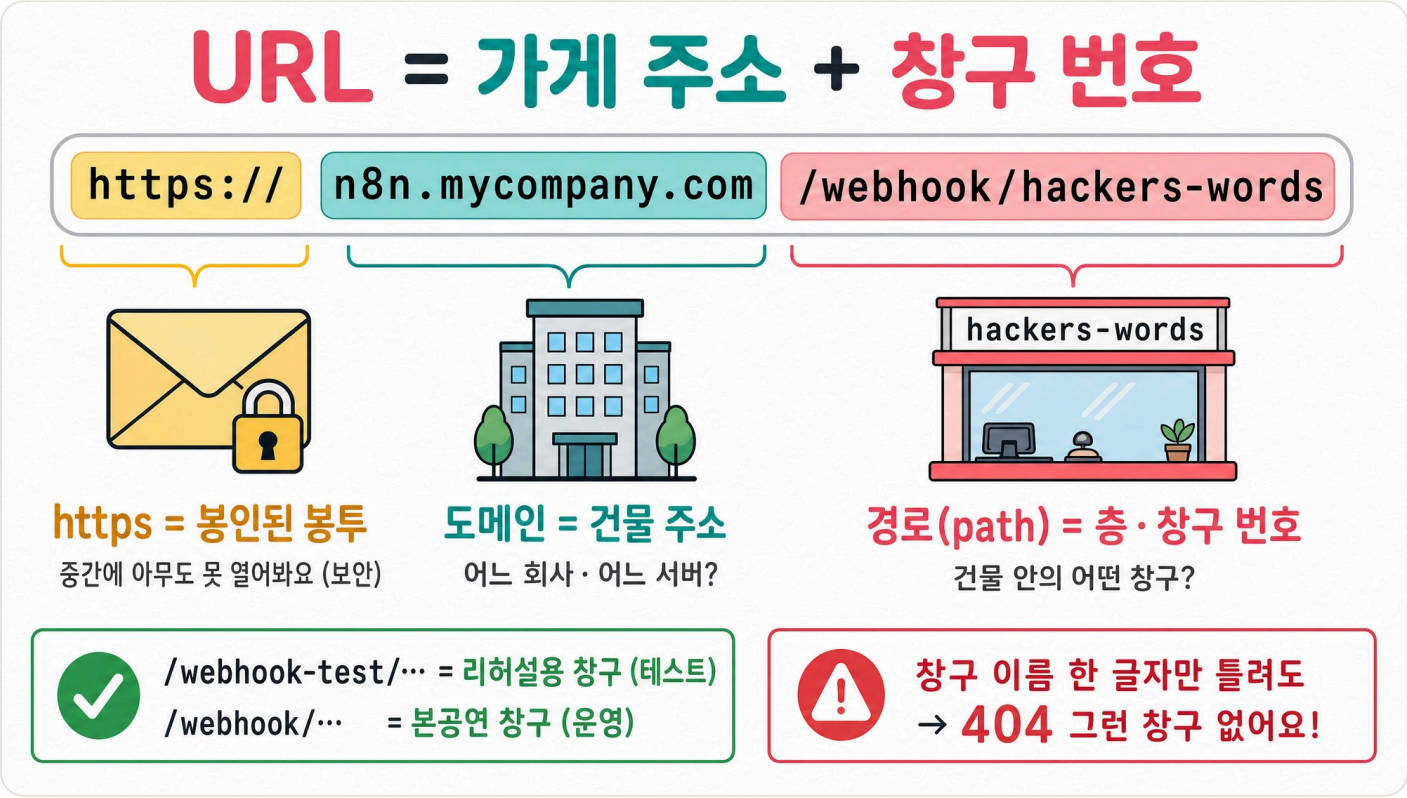
API = 식당 웨이터 — 손님(클라이언트)의 주문을 주방(서버)에 전달하고 요리(응답)를 가져온다

- 손님이 **주방에 들어가 요리법을 배울 필요가 없습니다.**
- **메뉴판(규칙)** 을 보고 **웨이터에게 주문**만 하면 요리(결과)가 나옵니다.
- **API = 프로그램끼리 대화하는 규칙**
    - ChatGPT, 카카오톡, 지도… 그리고 **n8n의 거의 모든 노드가 API로 움직입니다.**

> 💬 우리 연구원에서 “웨이터” 역할을 하는 사람은 누구일까요?

URL — 가게 주소와 테이블 번호 📮

API 를 부르려면 **어디로 주문할지 주소**가 필요합니다.

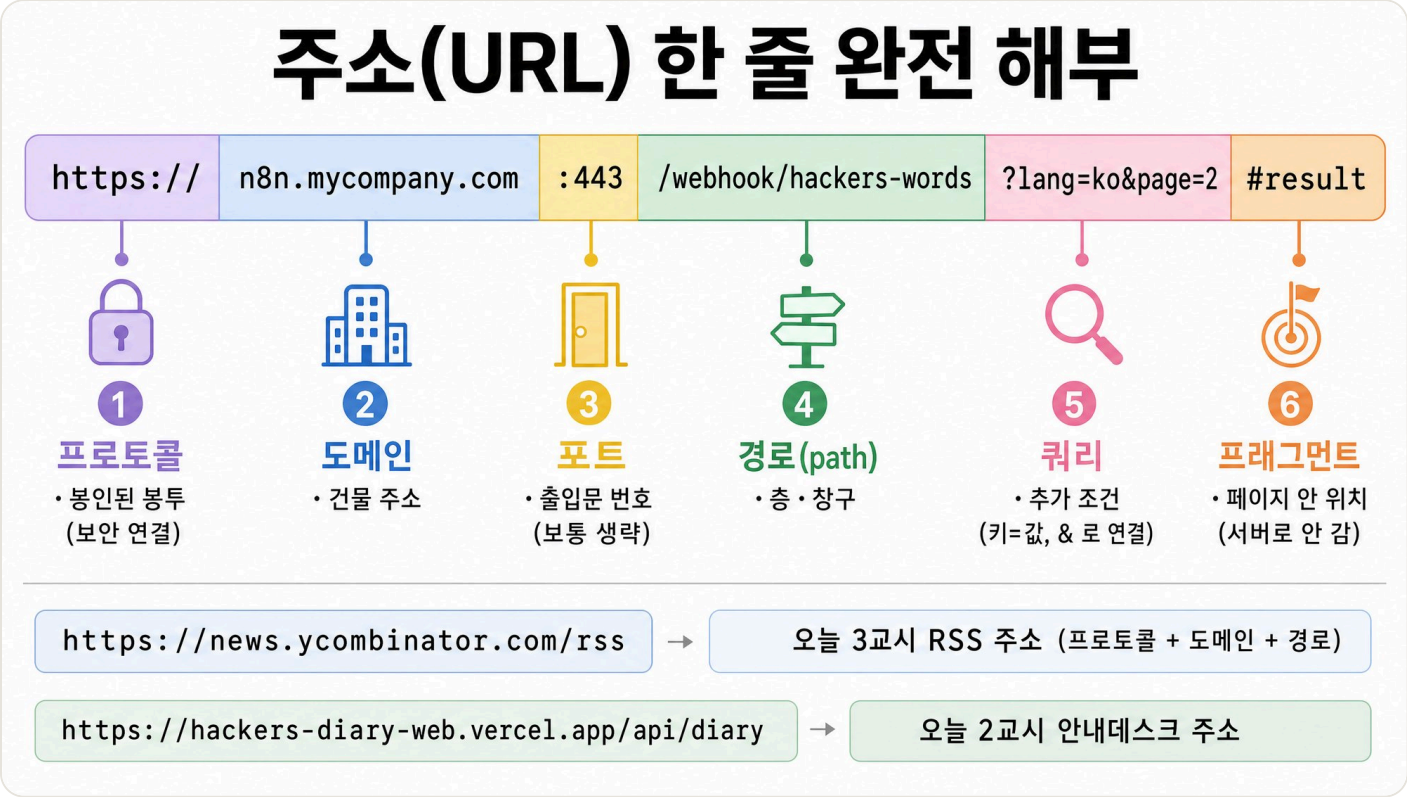


URL = 가게 주소 + 창구 번호

- **https** = 봉인된 봉투 (중간에 남이 못 열어봄)
- **도메인** = 건물 주소 — 어느 회사·서버인지

- **경로(path)** = 층·호수 — 그 안의 **어떤 창구**인지. 오늘 만들 창구 이름이 `hackers-words`
- 경로가 한 글자라도 틀리면 → **404 "그런 창구 없는데요?"**

주소 한 줄 완전 해부 (Full URL 예시)



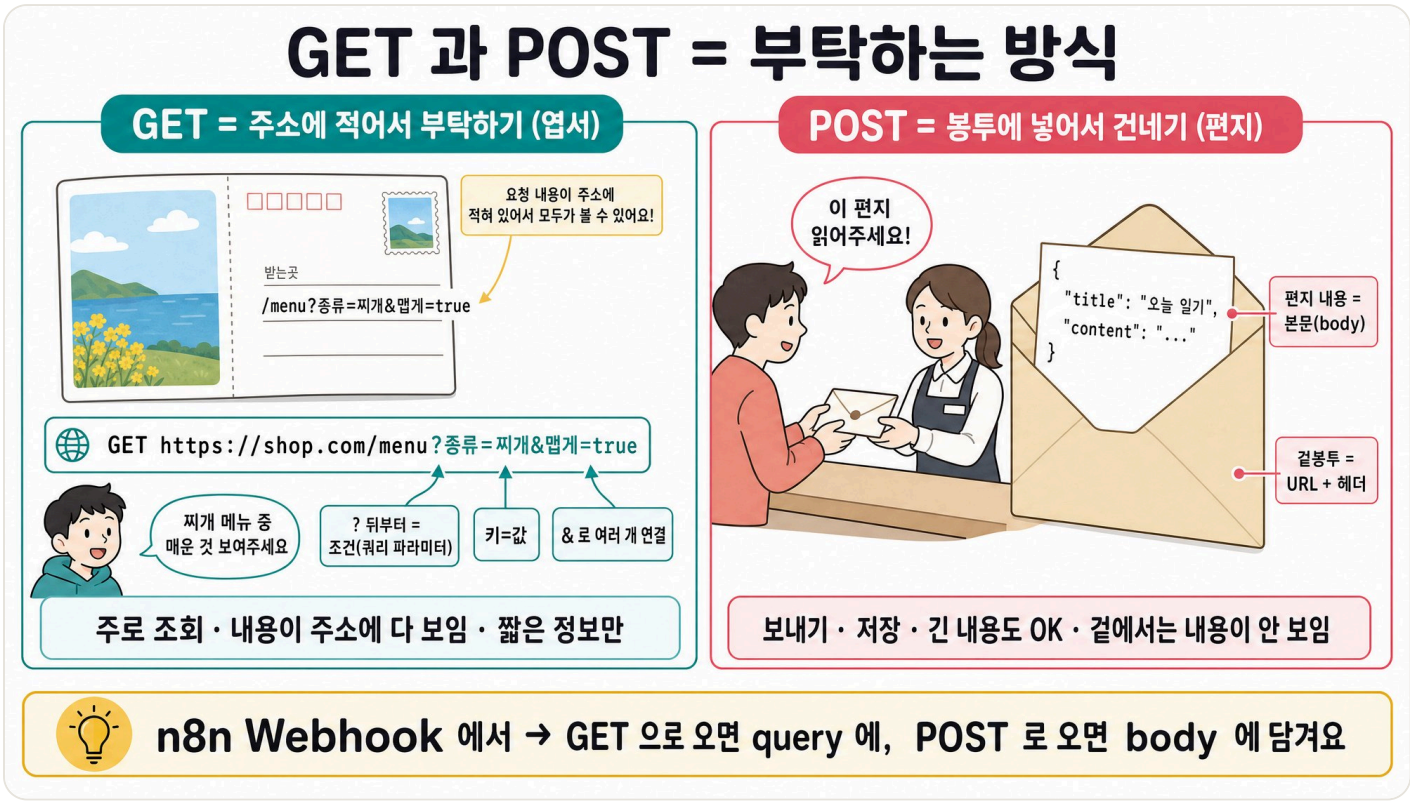
주소(URL) 한 줄 완전 해부 — 프로토콜 · 도메인 · 포트 · 경로 · 쿼리 · 프래그먼트

```
https://n8n.mycompany.com:443/webhook/hackers-words?lang=ko&page=2#result
└─①─┘ └──────②──────┘└③┘└──────④──────┘└──────⑤──────┘└─⑥─┘
```

번호	부분	예시	비유 · 설명
①	프로토콜	`https://`	봉인된 봉투 — 보안 연결
②	도메인	`n8n.mycompany.com`	건물 주소 — 어느 서버?
③	포트	`:443`	출입문 번호 — https 는 443 이 기본이라 **보통 생략**
④	경로	`/webhook/hackers-words`	층 · 창구 — 서버 안의 어느 기능?
⑤	쿼리	`?lang=ko&page=2`	추가 조건 — `?` 로 시작, `키=값` , 여러 개는 `&`
⑥	프래그먼트	`#result`	페이지 안의 위치(책갈피) — **서버로는 안 보내짐**

> 💡 매일 쓰는 주소에도 다 들어 있습니다. 네이버 검색 결과 주소 `https://search.naver.com/search.naver?query=n8n` 의 `?query=n8n` 이 바로 **쿼리** 예요.

주문 방식(메서드) — 엽서 vs 편지 ✉️



GET 과 POST = 부탁하는 방식 — 주소에 적는 엽서 vs 봉투에 넣는 편지

메서드	비유	내용은 어디에?	주로
GET	📮 **엽서** — 주소란에 부탁을 **적어서** 보냄	**URL 쿼리** `?종류=찌개&맵게=true` (누구나 보임)	조회 · 검색
POST	✉️ **편지** — "이 편지 읽어주세요!" **봉투 안에** 넣어 건넴	**본문(body)** 에 JSON (겉에서 안 보임)	보내기 · 저장 ⭐ 오늘의 주인공
PUT / DELETE	주문 변경 / 취소	본문 / URL	수정 / 삭제

같은 부탁을 두 방식으로 써 보면 👇

```
### GET — 부탁 내용이 주소에 그대로 보임
GET https://n8n.mycompany.com/webhook/hackers-words?word=고양이&word2=커피

### POST — 주소는 창구까지만, 부탁 내용은 봉투(body) 안에
POST https://n8n.mycompany.com/webhook/hackers-words
Content-Type: application/json

{ "words": ["고양이", "커피", "회의"] }
```

⚠️ 위 GET 줄은 **비교용 예시**입니다. 실습 웹훅은 POST 로 만들었기 때문에 GET 으로 부르면 404 가 납니다 (HTTP Method 를 GET 으로 바꾸면 동작).

🔎 **n8n Webhook 에서 확인하는 법**: GET 으로 오면 OUTPUT 의 `query` 칸에, POST 로 오면 `body` 칸에 담깁니다. 1교시 정답 화면에서 `query` 는 비어 있고 `body › words` 에 값이 있는 이유가 바로 이것!

영수증(상태코드) 🧾

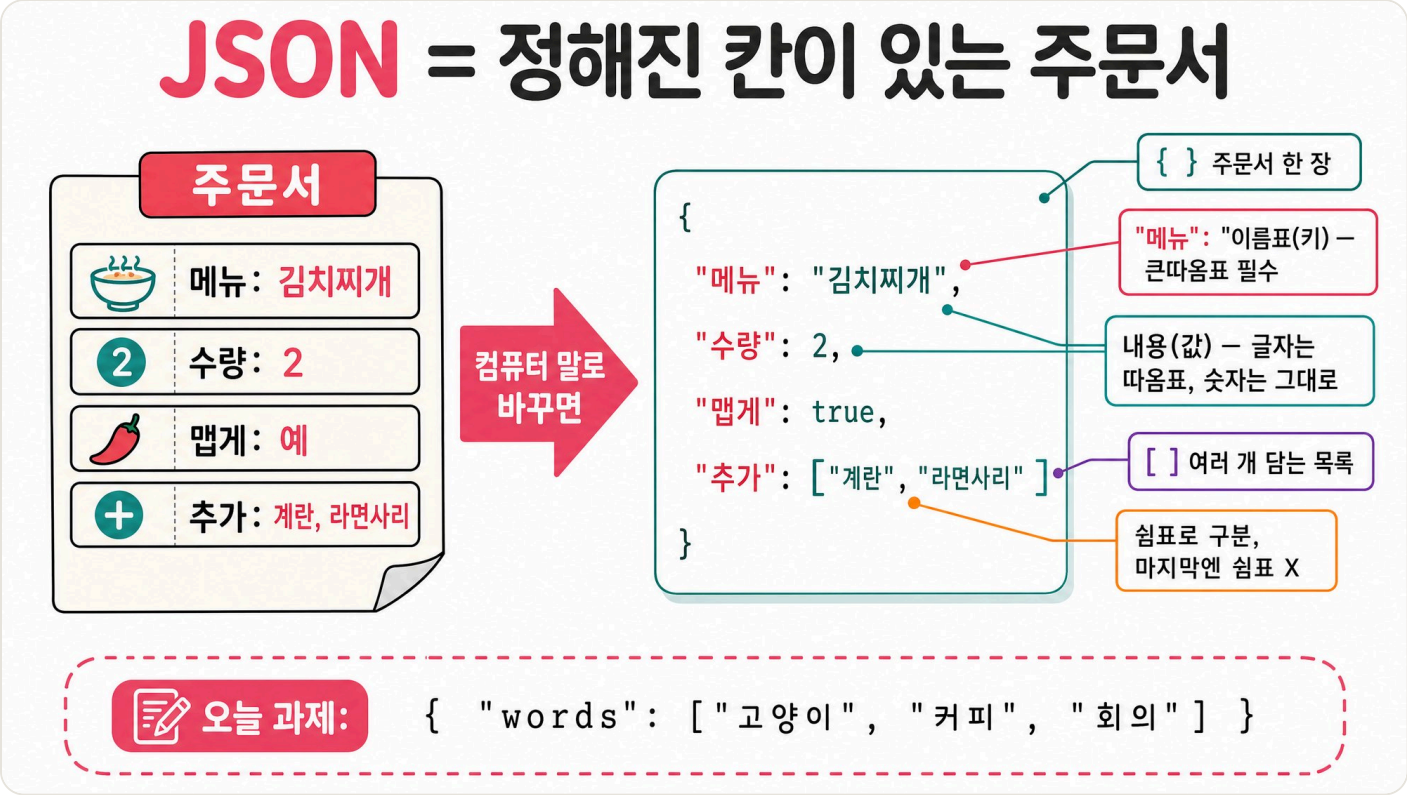


상태코드 = 주방이 돌려주는 영수증 — 200 · 404 · 500

상태코드	식당에서는	의미
200 (2xx)	“주문 잘 받았습니다”	✅ 성공
404 (4xx)	“그런 메뉴(주소) 없는데요?”	❌ **내 쪽 실수** — 주소 오타 · Listen 안 누름 · Publish 안 함 → **오늘 제일 많이 볼 오류**
500 (5xx)	“주방에 불났어요”	🔥 **서버 쪽 문제** — 잠시 후 다시 / 관리자에게

> 첫 숫자만 보면 됩니다: **2 = 성공, 4 = 내가 고칠 것, 5 = 서버가 고칠 것**

JSON — 주문서 양식



JSON = 정해진 칸이 있는 주문서

정해진 칸이 있는 주문서처럼 **이름표(키) : 내용(값)** 으로 적습니다.

```
{
  "메뉴": "김치찌개",
  "수량": 2,
  "맵게": true,
  "추가": ["계란", "라면사리"]
}
```

규칙	설명
`{ }`	주문서 한 장
`"키": 값`	이름표는 **반드시 큰따옴표**
`[ ]`	여러 개를 담는 **목록(배열)**
`,`	항목 구분 — **마지막 항목 뒤에는 쉼표 금지!**

오늘 과제 1의 주문서 👇 “words 라는 이름표에 단어 **3개짜리 목록**”

```
{ "words": ["고양이", "커피", "회의"] }
```

Webhook — 초인종



Webhook = n8n 에 달린 초인종 — 폴링 vs 웹훅, 그리고 동작 순서

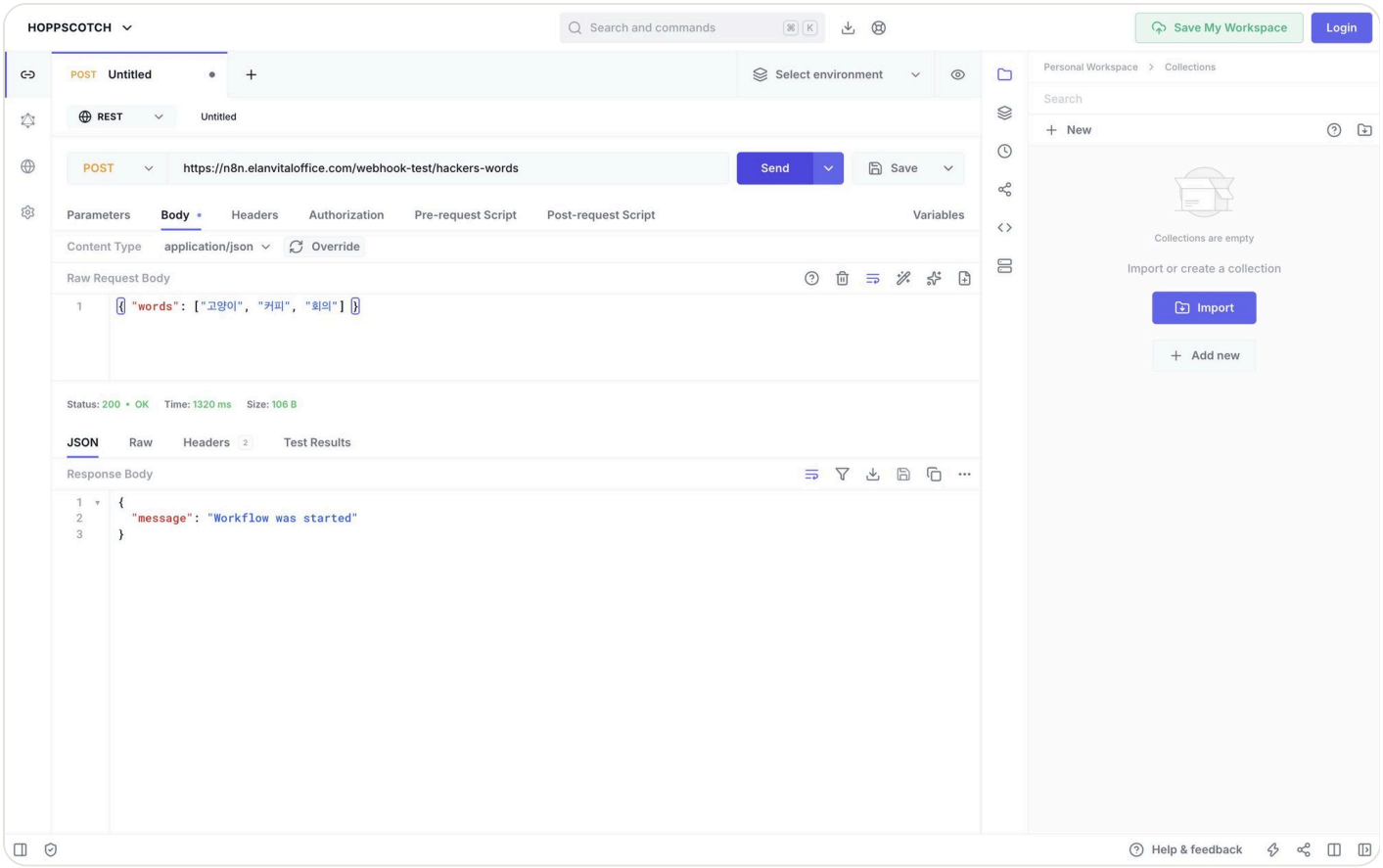
- 택배가 왔는지 **1분마다 문을 열어보는 대신**(= 폴링)
- 택배기사가 **초인종을 누르면 그때 반응**합니다 (= 웹훅)
- **n8n Webhook 노드 = n8n에 달린 초인종 + 우편함 주소**
    - 누군가 이 주소로 **JSON 주문서를 POST** 하면 → 워크플로우가 **자동 시작**

테스트 URL vs 프로덕션 URL = 리허설 vs 본공연

	🤹 **Test URL** `/webhook-test/…`	🎬 **Production URL** `/webhook/…`
언제	만들면서 확인할 때	완성 후 상시 운영
조건	**Listen for test event** 누른 뒤 **딱 1번**	워크플로우 **Publish(활성화)** 후 항상
결과	편집 화면 OUTPUT 에 바로 표시	Executions 탭에서 확인

Hoppscotch — 주문서를 보내는 우체통 📨

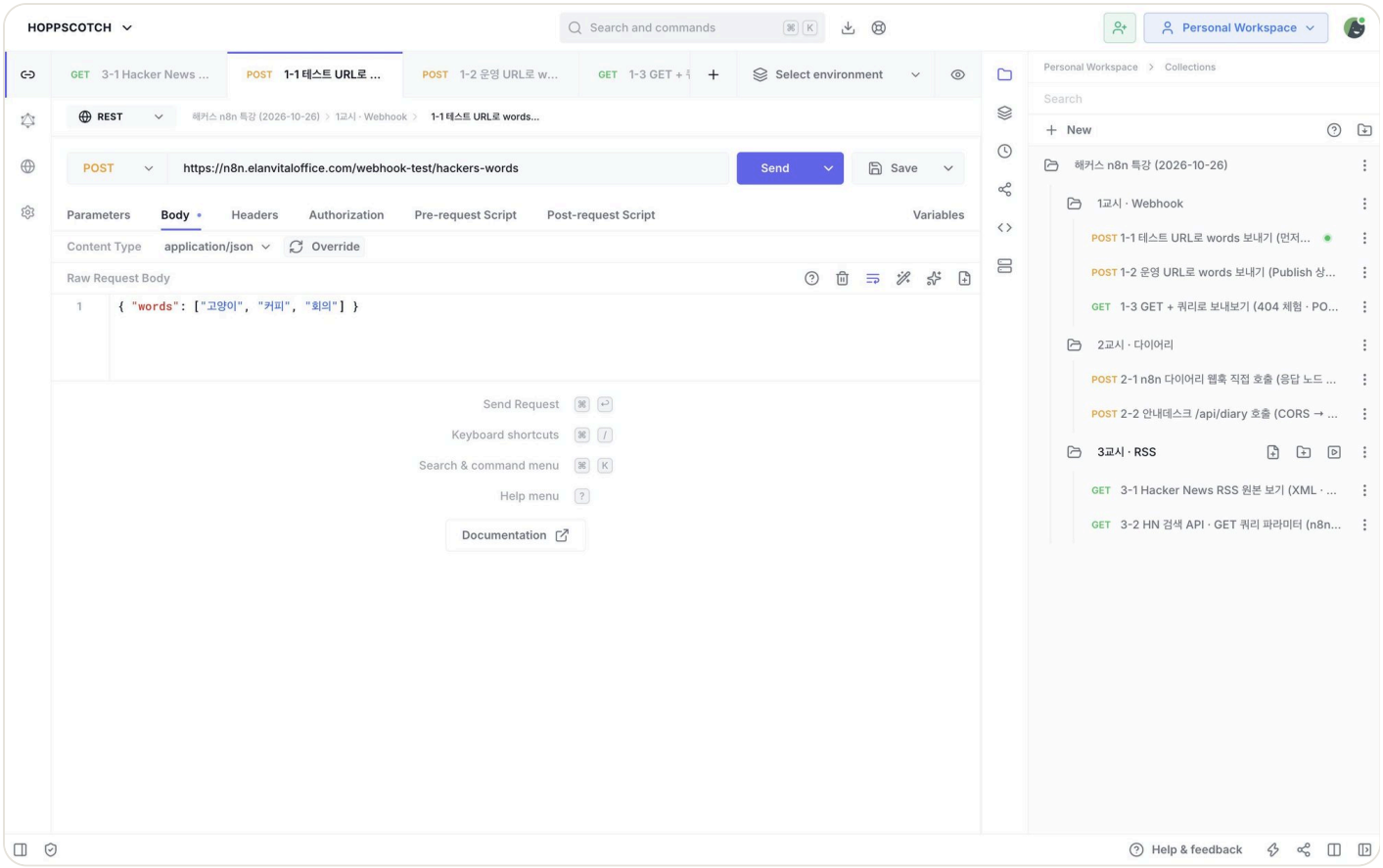
- 웹훅(초인종)을 만들었으면 **누군가 눌러 줘야** 합니다. 실습에서는 그 "누군가"를 우리가 직접 합니다.
- Hoppscotch = **설치 없이 브라우저에서** API 요청을 만들어 보내는 무료 도구 (Postman 의 웹 버전이라고 생각하세요)
- 화면 하나 = **주문서 한 장**: ① 방식(POST) ② 주소(URL) ③ 형식(Content-Type) ④ 내용(Body · JSON) → **Send** → 아래에 **영수증(상태코드)과 답장(응답)**



Hoppscotch 로 n8n 테스트 웹훅에 words 를 보낸 화면 — Status 200 OK

워크스페이스 · 컬렉션 — 주문서 서랍장 📦

- 구글 계정 등으로 **로그인**하면 내 요청이 **Personal Workspace(개인 작업 공간)** 에 저장되어 어느 PC 에서나 다시 꺼내 쓸 수 있습니다.
- **컬렉션** = 요청들을 담는 **서랍**, **폴더** = 서랍 속 칸막이. 오늘 강의는 `1교시 · Webhook` / `2교시 · 다이어리` / `3교시 · RSS` 폴더로 정리해 두었습니다.
- 컬렉션은 **Export(내보내기) → 파일 1개** 로 저장하고, 다른 사람은 **Import(가져오기)** 로 그대로 받아 쓸 수 있습니다 → 셀프 학습 키트에 넣어 드립니다 🎁



오늘 강의용 컬렉션 — 교시별 폴더 7개 시나리오

> 💻 Cursor / VS Code 를 쓰는 분은 확장 **REST Client** 의 `.http` 파일로도 똑같이 보낼 수 있습니다. 터미널 `curl`, Postman, 웹페이지의 버튼(2교시!)도 모두 같은 일을 합니다.

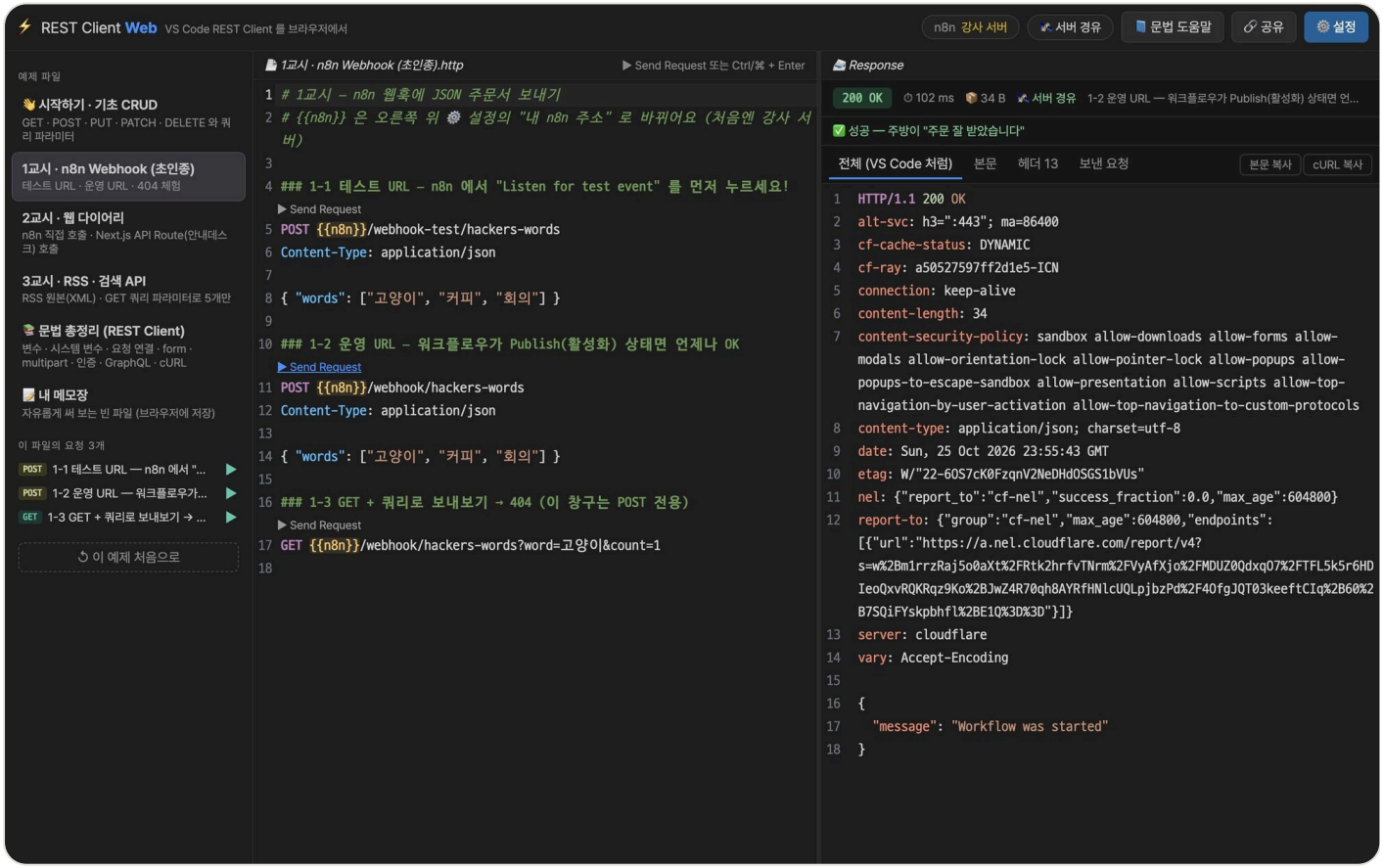
📦 셀프 학습 키트 · 보내는 도구

- **hackers-n8n-selfstudy.json** — 오늘 시나리오 7개 Hoppscotch 컬렉션 (Import 용) · **가져오는 방법**
- **webhook.http** (Cursor REST Client) · **curl.sh** (Mac/Linux) · **curl.ps1** (Windows)

⚡ REST Client Web — VS Code 문법 그대로, 브라우저에서

⚡ hackers-rest-client.vercel.app — 강의 전용 웹 REST Client (설치 · 로그인 없음)

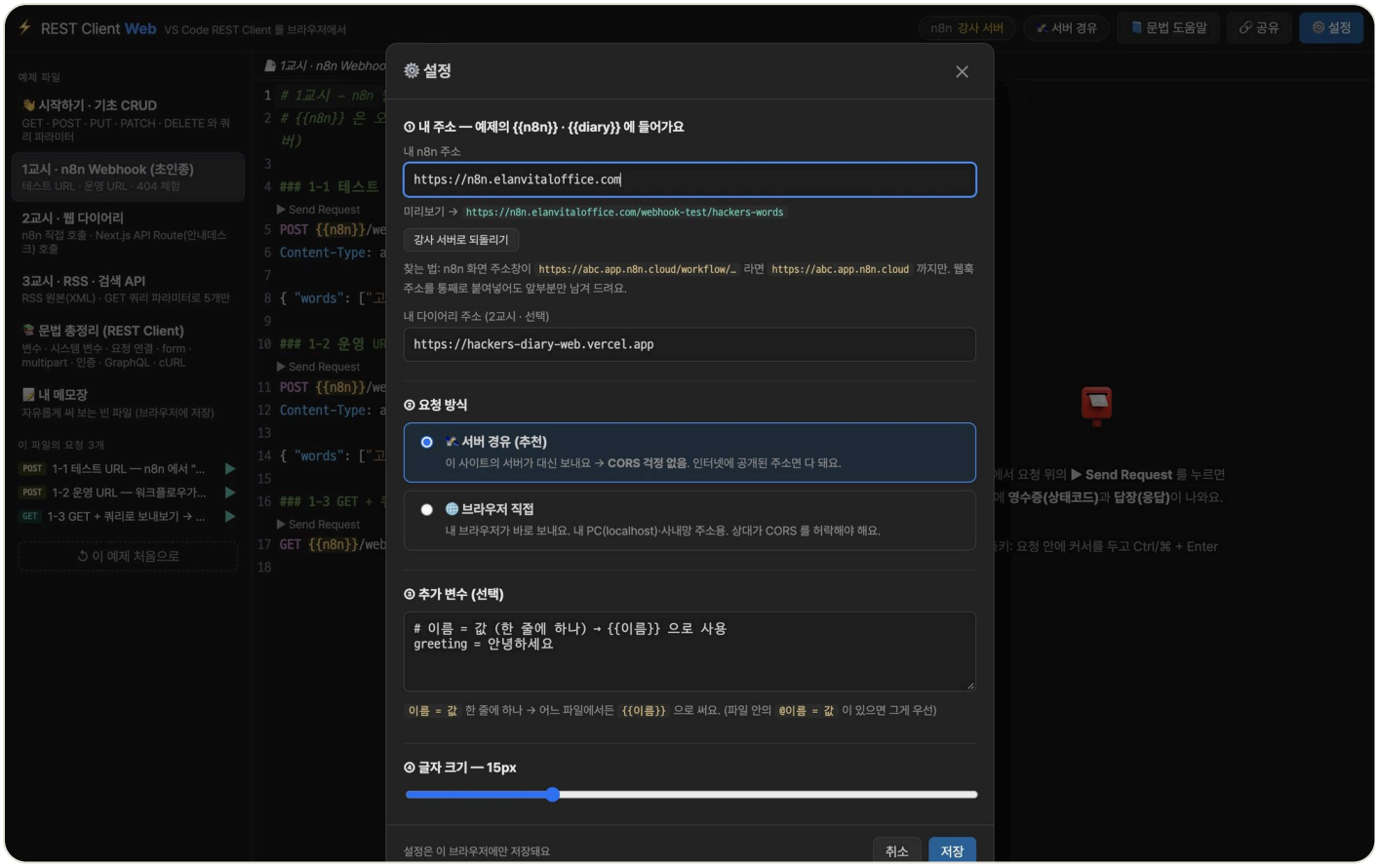
- VS Code 확장 **REST Client** 의 `.http` 문법을 **브라우저에서 그대로** — 요청 위 ▶ **Send Request** 를 누르면 오른쪽에 영수증과 답장
- **CORS 걱정 없음**: 우리 서버(서울)가 대신 보내 줘서 RSS · 남의 API · Vercel 사이트 모두 바로 됩니다 (Proxy 설정 불필요)
- **오늘 예제 내장**: 왼쪽 목록에서 `1교시 · n8n Webhook` → `1-1` 의 ▶ Send Request



REST Client Web — 1교시 예제를 보낸 화면

⚙️ 내 n8n 주소는 설정에서 한 번만

오른쪽 위 ⚙️ **설정 → 내 n8n 주소** 에 내 주소를 넣으면, 모든 예제의 `{{n8n}}` 이 내 주소로 바뀝니다. (웹훅 주소를 통째로 붙여넣어도 앞부분만 남겨 줘요 · 브라우저에 저장)



⚙️ 설정 — 내 n8n 주소 · 요청 방식 · 추가 변수

요청 방식	언제
🛰️ **서버 경유** (기본 · 추천)	인터넷에 공개된 주소 — n8n Cloud, RSS, 남의 API … CORS 와 무관
🌐 **브라우저 직접**	내 PC(localhost) · 사내망 n8n — 서버는 들어갈 수 없는 곳

> 📚 왼쪽 📚 문법 총정리 파일에 **변수 · 시스템 변수 · 요청 연결 · 폼 · multipart · 인증 · GraphQL · cURL** 예제가 모두 들어 있어요.
> 위에서부터 하나씩 Send!

실습 1 · 초인종 만들고 눌러보기

🎬 **따라하기 영상 · 과제 1 Webhook 초인종** — Webhook 노드 만들기 → 테스트 URL · 운영 URL 로 각각 확인 · JSON 주문서 읽는 법 포함 (약 4분 25초) — 영상은 강의자료 웹(hackers-n8n-lecture.vercel.app)에서 재생

📦 **셀프 학습 키트 · 실습 1 워크플로우**

- 01_webhook_words.json — 노드마다 한국어 설명 메모 포함
- n8n 에서 ⋯ → **Import⋯** → **Import from URL⋯** 에 붙여넣기: `https://raw.githubusercontent.com/joosung80/hackers-n8n-selfstudy-kit/main/workflows/01_webhook_words.json`
- 📘 매뉴얼 2장 — 같은 실습을 혼자 다시 하기

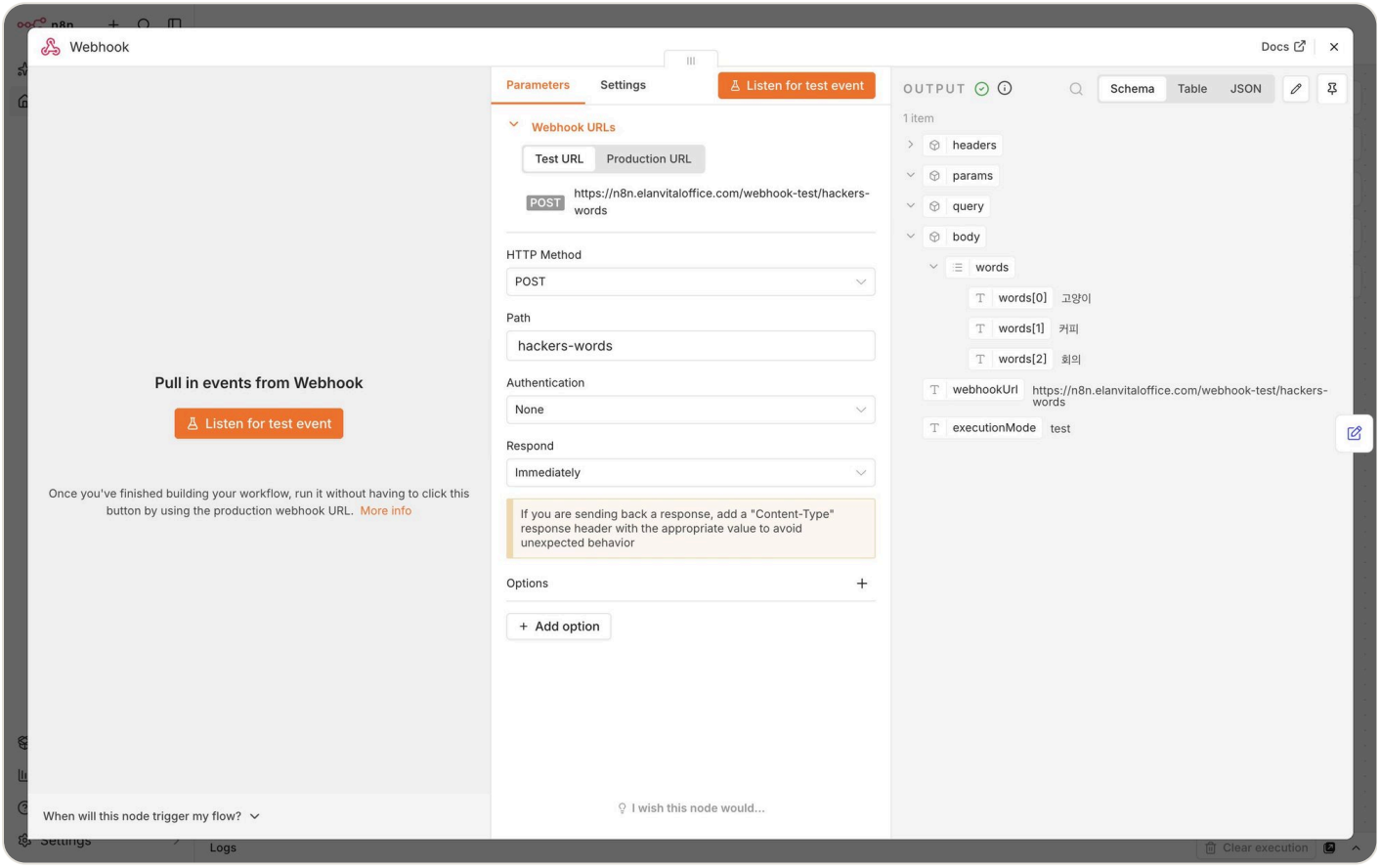
1. n8n → **Create workflow** → `+` → **Webhook**
2. **HTTP Method: POST** · **Path:** `hackers-words`
3. Webhook URLs 에서 **Test URL** 복사
4. 브라우저에서 hackers-rest-client.vercel.app 의 `1-1` (⚙️ 설정에 내 n8n 주소) — 또는 hoppscotch.io 열기 (컬렉션이 있으면 `1-1 테스트 URL로 words 보내기` 클릭)
   - 방식 **POST** · 주소칸에 3번의 **Test URL** 붙여넣기
   - **Body** 탭 → Content Type **application/json** → 아래 내용 입력

```
{ "words": ["고양이", "커피", "회의"] }
```

5. n8n 에서 **Listen for test event** 클릭 (🔔 초인종 켜기) — **반드시 Send 보다 먼저!**
6. ▶ **Send Request** (Hoppscotch 는 **Send**) → 아래에 `Status: 200 OK` · `{"message": "Workflow was started"}`

7. OUTPUT → **Schema** 탭 → `body › words` 에서 **고양이 · 커피 · 회의** 확인
8. 📸 **캡처!**

> 💡 **headers** = 택배 상자에 붙은 송장 · **body** = 상자 안의 내용물



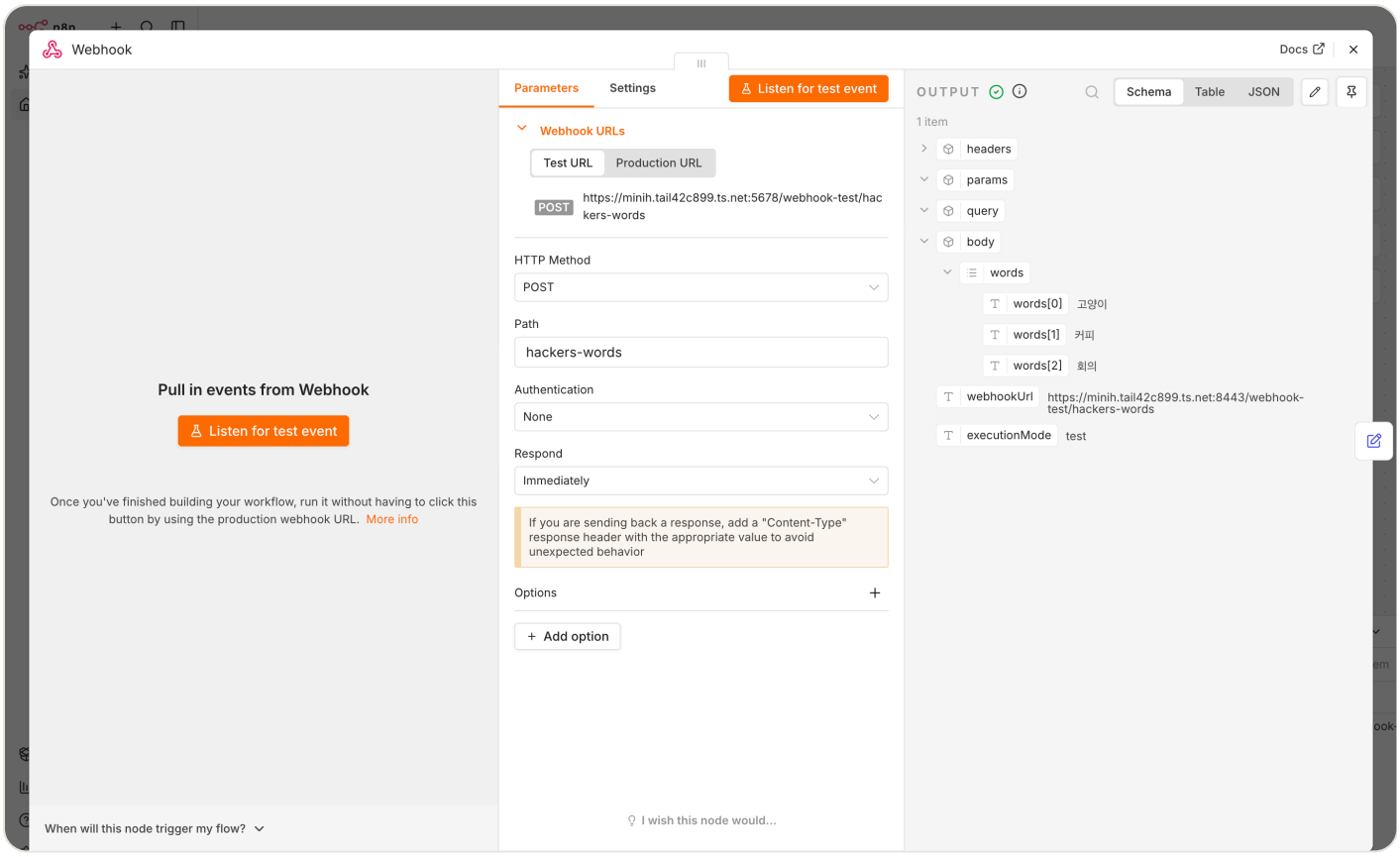
n8n 쪽 — Hoppscotch 가 보낸 words 3개가 body 에 도착

> 📝 과제 1 문항은 “Cursor 의 REST Client” 로 쓰여 있지만, 채점은 **n8n Webhook OUTPUT 화면** 으로 합니다. Hoppscotch · REST Client 어느 쪽으로 보내도 됩니다.

자주 막히는 곳

증상	해결
`404 … webhook is not registered`	**Listen for test event** 를 다시 누르고 재전송 (테스트 URL은 1번만 받음)
words 가 안 보이거나 한 줄 글자로 보임	Hoppscotch **Body → Content Type 이 application/json** 인지 (REST Client 는 `Content-Type` 줄 · **빈 줄**)
JSON 오류	마지막 쉼표, 작은따옴표 → **큰따옴표** 로
Hoppscotch 에 `Network Error`	주소 오타 확인 → 그래도면 **CORS** (아래 팁) → Settings → Interceptor **Proxy**
(REST Client) Send Request 버튼이 없음	REST Client 확장 설치 · 파일 확장자 `.http` 확인

✅ 문제 1 정답 화면



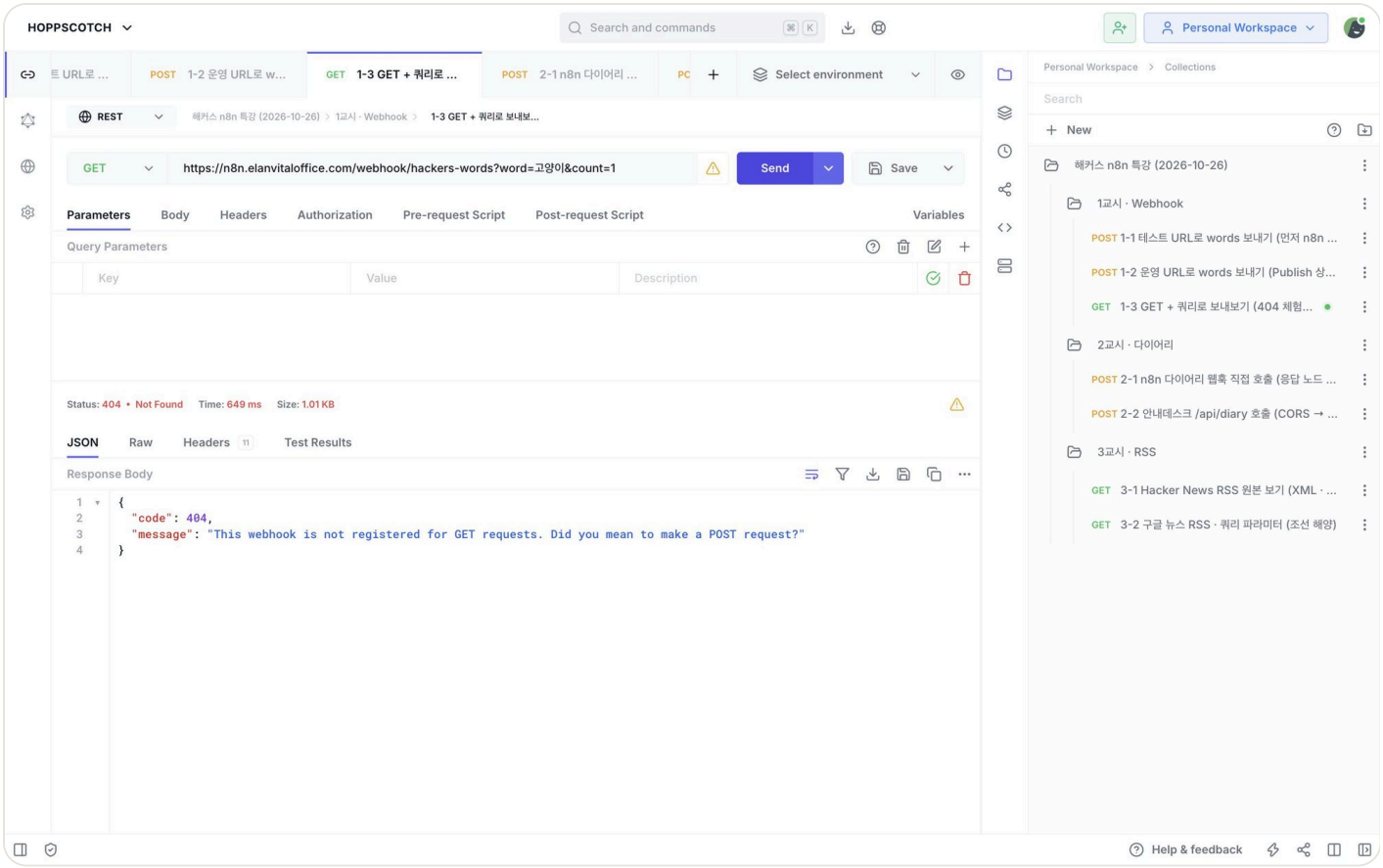
문제 1 정답 — Webhook OUTPUT 에 words 3개

- [ ] Webhook 실행 성공, OUTPUT 이 보인다 **(10점)**
- [ ] `words` 안에 값 **3개** 가 보인다 **(20점)**

> 📸 팁: Schema 탭에서 **headers 를 접으면** words 가 한 화면에 들어옵니다.

더 해보기 · GET 으로 보내면? (404 체험)

컬렉션의 `1-3` 은 같은 창구에 **GET + 쿼리** (`?word=고양이&count=1`) 로 보내 봅니다. 우리 웹훅은 **POST 전용 창구**라서 → **404 "그런 창구 없어요"**. 상태코드 영수증을 눈으로 확인하는 시나리오입니다.

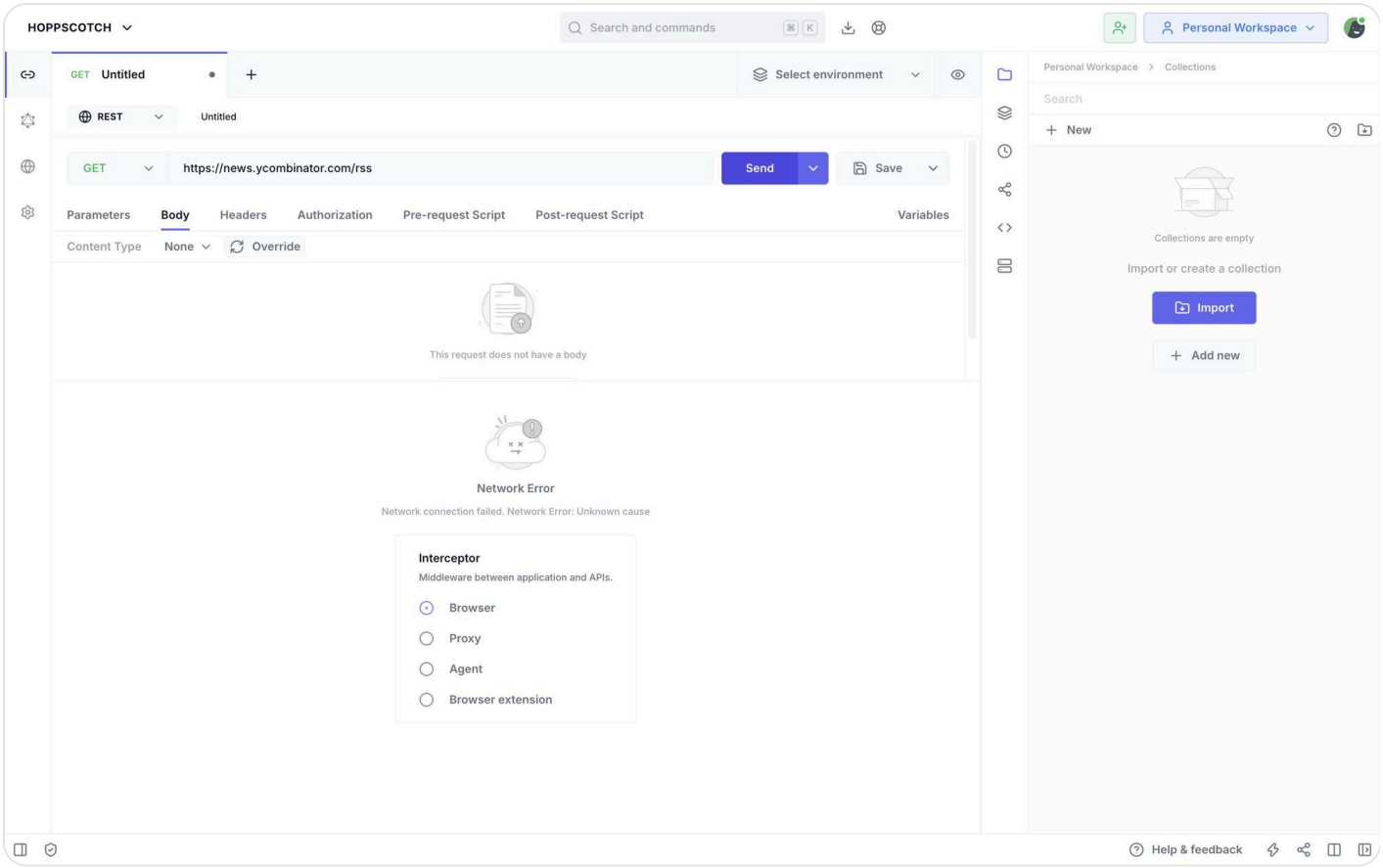


1-3 GET 으로 보내면 404 Not Found

💡 팁 · CORS 와 Proxy — 브라우저 도구의 규칙

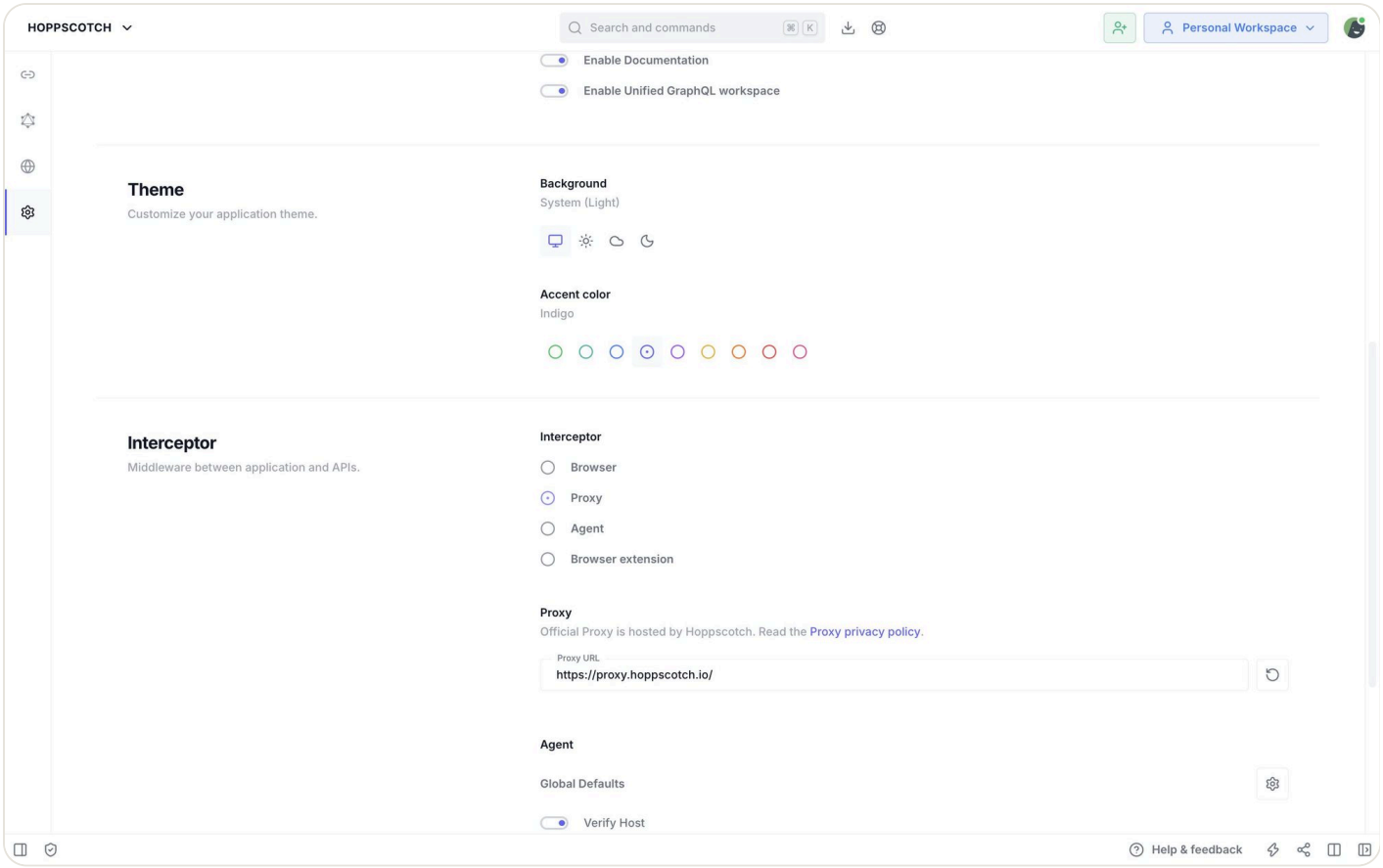
> **브라우저에서 동작하는 도구(Hoppscotch)로 다른 사이트를 부르려면, 상대 서버가 "브라우저에서 와도 돼요" 라고 허락(CORS)해 줘야 합니다.**

- **CORS** = 상대 서버가 **"이 사이트(origin)에서 오는 브라우저 요청은 받아줄게"** 라고 적어 주는 **출입 허가증**
- **n8n 웹훅은 허락해 줍니다** → 그래서 1교시는 바로 됐어요. (n8n 이 받은 headers 에 `origin: https://hoppscotch.io` 가 찍혀 있습니다)
- **Hacker News RSS, 우리 Vercel `/api/diary` 는 허락이 없습니다** → 브라우저가 막아서 `Network Error`



CORS 차단 — Hacker News RSS 를 Browser 모드로 부르면 Network Error

해결: Proxy(대리인) 모드 1. Hoppscotch 왼쪽 ⚙️ **Settings** → **Interceptor** → **Proxy** 선택 (또는 오류 화면의 Interceptor 상자에서 바로 Proxy) 2. 다시 **Send** → Hoppscotch 의 서버(`proxy.hoppscotch.io`)가 **대신 요청**해 주므로 CORS 와 무관하게 성공



Settings → Interceptor → Proxy

모드	누가 요청하나	언제
Browser (기본)	내 브라우저	상대가 CORS 를 열어 둔 API (n8n 웹훅 등)
Proxy	Hoppscotch 서버가 대신	CORS 가 없는 사이트 (RSS, 남의 API)
Agent / Extension	내 PC 에 설치한 도우미	회사 내부망 · localhost 주소

⚠️ Proxy 는 요청이 **Hoppscotch 서버를 거쳐 갑니다** → 비밀번호 · API 키 · 개인정보가 담긴 요청에는 쓰지 마세요. 실습이 끝나면 **Browser** 로 되돌려 두세요.

💡 **n8n · 2교시 API Route 는 서버라서 CORS 와 상관없습니다.** CORS 는 오직 **브라우저** 의 규칙입니다. 그래서 2교시에 "브라우저 → 우리 서버 (/api/diary) → n8n" 구조가 안전하고 편합니다.

2교시 · 웹 다이어리 + Vercel 배포

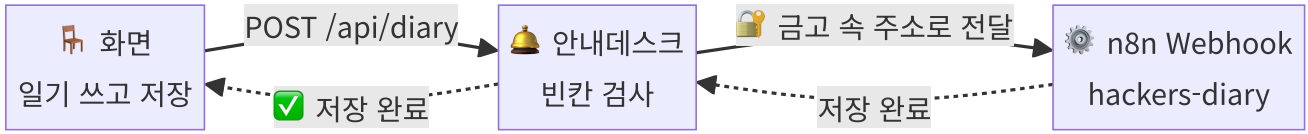
📚 이번 교시 필수 개념 — 딱 4개

🪑 화면 (프론트엔드) 🛎️ 안내데스크 (API) 🔐 금고 (환경변수)

🏨 개업 (Vercel 배포)

목표: **내 다이어리 웹이 인터넷에 열리고, 거기서 쓴 일기가 n8n 에 도착**

한 장으로 보기



비유	실제	하는 일
🪑 식당 홀	화면 (브라우저)	입력칸 · 저장 버튼
🛎️ 안내데스크	API `/api/diary` (서버)	빈칸 검사 → n8n 에 전달
🔐 금고	환경변수 `N8N_WEBHOOK_URL`	n8n 주소를 숨겨 둠
🏨 개업	Vercel 배포	`…vercel.app` 인터넷 주소가 생김

> 🧠 오늘 기억할 것은 **이 표 하나**입니다. 아래는 이 표를 한 칸씩 확대해 보는 것뿐이에요.

왜 안내데스크를 거칠까?

- 화면 코드에 n8n 주소를 쓰면 **F12 한 번에 주소가 보임** → 아무나 호출 (장난 · 스팸)
- 안내데스크는 **서버에서만** 실행 → 주소는 금고에 두고, 손님은 `/api/diary` 만 압니다
- 덤: 서버끼리 통신이라 **CORS 오류도 없어요**

코드는 파일 2개가 핵심

과제는 **Next.js** 로 만듭니다. 파일은 여러 개지만 우리가 볼 것은 **두 개**뿐이에요.

```
diary-web/                          (Next.js 템플릿)
├── app/page.tsx                    🪑 화면 — 저장 버튼을 누르면 fetch('/api/diary')
└── app/api/diary/route.ts          🔔 안내데스크(API Route) — 주소 /api/diary
```

안내데스크 `route.ts` 를 다섯 단계로 줄이면:

```
export async function POST(request) {
  // ① 주문서 받기
  const { title, content } = await request.json()
  // ② 빈칸 검사
  if (!title || !content) return Response.json({ ok: false }, { status: 400 })
  // ③ 금고에서 n8n 주소 꺼내기
  const url = process.env.N8N_WEBHOOK_URL
  // ④ n8n 으로 전달
  await fetch(url, { method: 'POST', body: JSON.stringify({ title, content }) })
  // ⑤ 손님에게 답장
  return Response.json({ ok: true, message: '저장 완료' })
}
```

📦 **전체 코드 (주석 포함)**

- app/page.tsx · app/api/diary/route.ts

- 📎 참고: Next.js 없이 같은 구조를 파일 2개로 만든 **심플 버전** (`index.html` + `api/diary.js`) — 과제 제출용은 아니에요

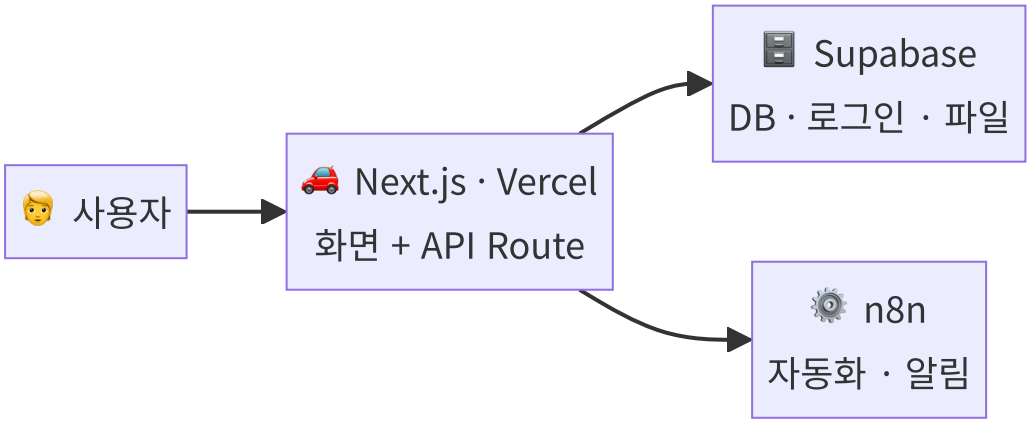
Next.js 는 뭐예요?

Next.js = 화면 + 백엔드를 한 프로젝트에 담는 웹사이트 조립 키트. Vercel 회사가 만들어서 Vercel 과 궁합이 가장 좋아요. (React 가 **엔진**이라면 Next.js 는 엔진을 얹은 **완성차**)

왜 Next.js 일까? — 백엔드가 "폴더 하나" 로 끝나요

- 보통 웹 서비스는 **화면 프로젝트 + 백엔드 서버(Express · Spring 등)** 를 따로 만들고, 따로 배포하고, 따로 관리합니다.
- Next.js 는 `app/api/diary/route.ts` 파일 하나 만들면 그게 곧 백엔드 주소 `/api/diary` → **백엔드 라우팅(API Route)이 내장**
- 화면과 백엔드가 **한 저장소 · 한 언어 · 한 번 배포** — 오늘 만든 안내데스크가 바로 이것

나중에 · Next.js + Supabase = 복잡한 백엔드 없이 진짜 서비스



필요한 것	예전 방식	Next.js + Supabase
화면	프론트 프로젝트 따로	`app/page.tsx`
API 서버	백엔드 서버 직접 구축 · 운영	`app/api/…/route.ts` 파일
DB · 로그인 · 파일 저장	DB 설치, 회원가입 기능 직접 개발	**Supabase** 가 대신 (클릭 몇 번)
서버 관리	서버 임대 · 보안 패치	Vercel · Supabase 가 알아서

> 예: 다이어리에 **"내 일기 목록 · 로그인"** 을 붙이고 싶다면 → API Route 에서 Supabase 에 저장 · 조회 몇 줄, 요약 · 알림은 n8n 이 담당. **백엔드 서버를 따로 세우지 않아도 됩니다.**

🚗 Next.js vs 🚲 심플 — 과제는 Next.js, 심플은 참고

	🚗 Next.js 버전 · 과제	🚲 심플 버전 · 참고
화면	`app/page.tsx`	`index.html`
안내데스크	`app/api/diary/route.ts`	`api/diary.js`
파일 수	6개 + 설정 파일	2개
어울리는 때	페이지 · API 가 늘고 **로그인 · DB(Supabase)** 를 붙여 서비스로 키울 때	페이지 1~2장 · 구조 이해용

- 규칙은 같아요: **폴더 위치 = 웹 주소** (`api/diary` → `/api/diary`), 안내데스크 다섯 단계(①~⑤)도 똑같음
- 차이: 심플의 `api/` 폴더는 **Vercel 이 주는 기능**, Next.js 의 API Route 는 **프레임워크 자체 기능** — 페이지 · API 가 수십 개로 늘어도 같은 규칙으로 정리돼요

개업 준비물: 금고 · 창고 · 본사

	비유	오늘 할 일
🔐 환경변수	금고	`N8N_WEBHOOK_URL` 에 n8n Production URL 넣기
🗄️ GitHub	코드 창고	버튼 한 번에 내 창고로 복사
🏨 Vercel	프랜차이즈 본사	창고 코드를 받아 개업 · 코드가 바뀌면 자동 재개업

> ⚠️ Next.js 에서 환경변수 이름 앞에 `NEXT_PUBLIC_` 을 붙이면 **유리 금고** (브라우저에 다 보임)가 됩니다. 오늘은 절대 붙이지 않아요.

실습 2-1 · n8n 에 다이어리 초인종 달기

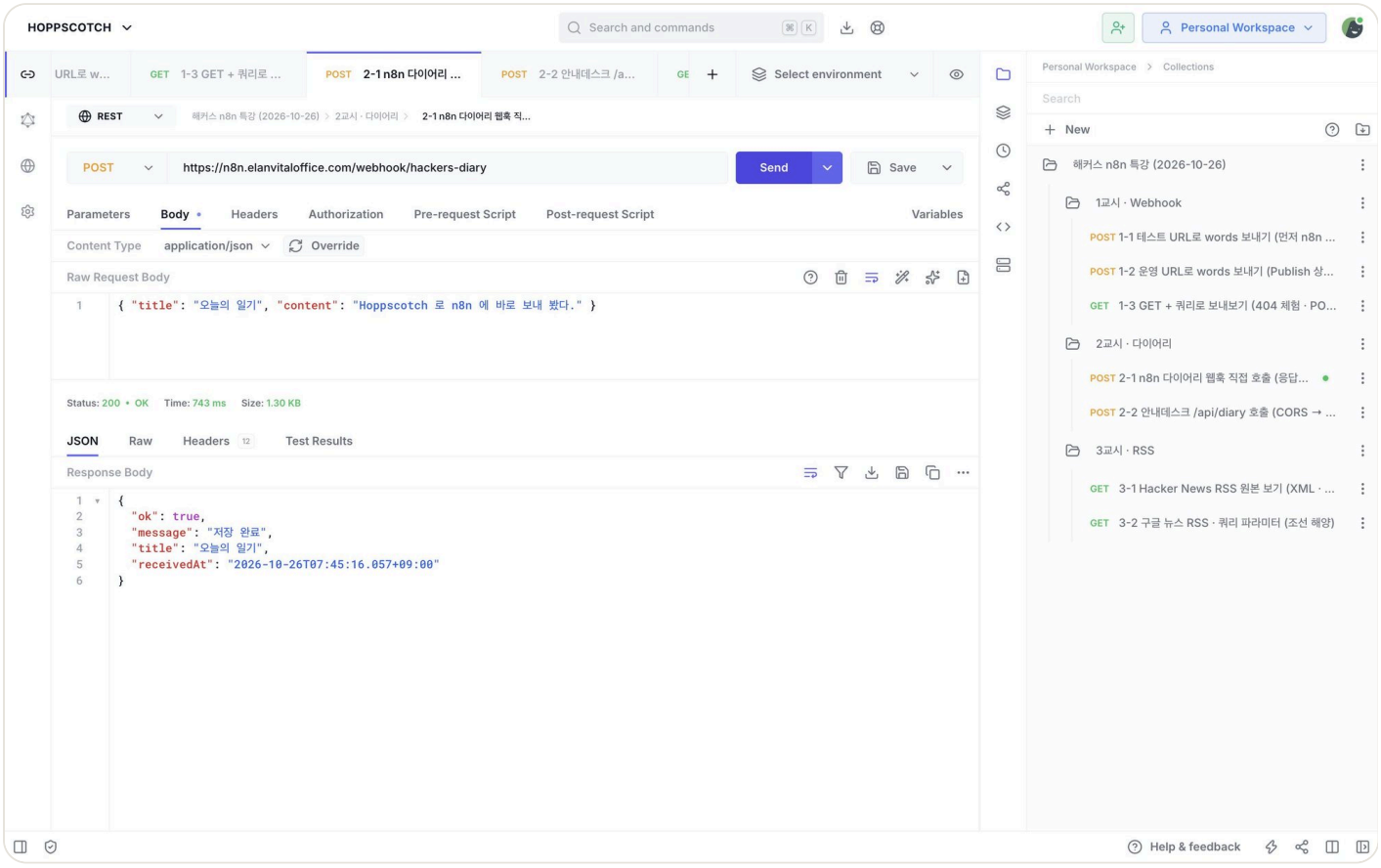
🎬 **따라하기 영상 · 과제 2 웹 다이어리** — 빈 캔버스에서 노드를 하나씩 만들어 가는 과정 (약 2분 40초) — 영상은 강의자료 웹(hackers-n8n-lecture.vercel.app)에서 재생

📦 **셀프 학습 키트 · 실습 2 워크플로우**

- 02_diary_webhook.json — Import from URL: `https://raw.githubusercontent.com/joosung80/hackers-n8n-selfstudy-kit/main/workflows/02_diary_webhook.json`

1. `02_diary_webhook.json` **Import** (가장 빠름) — 또는 직접: **Webhook** 하나 (**POST** · Path `hackers-diary` · Respond 는 기본값 *Immediately*)
2. 오른쪽 위 **Publish**
3. **Production URL** 복사 — `/webhook/hackers-diary` (⚠️ `webhook-test` 아님!)

4. (확인) REST Client Web 또는 Hoppscotch 의 `2-1` → **Send** → `200` + `Workflow was started`



2-1 n8n 다이어리 웹훅에 직접 보내 보기

실습 2-2 · 버튼 한 번으로 개업하기

과제는 **Next.js 버전**으로 제출합니다 👉

▲ Deploy

> 📎 참고: Next.js 없이 만든 🚲 심플 버전 도 같은 방법으로 배포할 수 있어요 (구조 이해용 · 과제 제출용 아님).

1. **Deploy** 버튼 → GitHub 로 로그인 → 저장소 이름 그대로 **Create**

2. **N8N_WEBHOOK_URL** 칸에 2-1 에서 복사한 **Production URL** 붙여넣기
3. **Deploy** → 1분쯤 뒤 🎉 → 화면의 **도메인** 클릭
4. 제목 · 내용 쓰고 **저장하기** → ✅ **저장 완료** → n8n **Executions** 에서 도착 확인
5. 📸 **브라우저 창 전체 캡처** — 주소창 · 입력칸 · 저장 버튼 · 저장 완료가 **한 화면에**

실습 2-3 (선택) · ChatGPT 에게 직접 만들어 달라고 하기

길게 설명할 필요 없어요. **목적 · 보낼 것 · 주소 · 금고** 네 가지만 말하면 됩니다.

🚗 Next.js 버전 (과제용)

```
Next.js 로 웹 다이어리를 만들어서 Vercel 에 올리려고 해.
제목(title)과 내용(content)을 저장하면 아래 n8n 주소로 보내지면 돼.
n8n 주소는 Vercel 환경변수로 설정할 거야. Vercel 에서 어떻게 설정하는지도 알려줘.
Next.js 는 최신 버전(latest)으로 해 줘.
n8n 주소: https://(내 n8n 주소)/webhook/hackers-diary
```

🚲 **심플 버전 (참고)** — 첫 줄의 “Next.js 로” 와 “최신 버전” 줄을 빼면 끝

```
웹 다이어리를 만들어서 Vercel 에 올리려고 해.
제목(title)과 내용(content)을 저장하면 아래 n8n 주소로 보내지면 돼.
n8n 주소는 Vercel 환경변수로 설정할 거야. Vercel 에서 어떻게 설정하는지도 알려줘.
n8n 주소: https://(내 n8n 주소)/webhook/hackers-diary
```

문장	왜 넣나
웹 다이어리 · Vercel	**목적** — 무엇을, 어디에 올릴지
title · content → n8n 주소	**보낼 것** — 1교시의 "이름표" 그대로
환경변수로 설정할 거야	**금고** — 이 한마디에 AI 가 알아서 안내데스크(`/api/diary`)를 만들어 줘요
최신 버전	옛 버전이면 Vercel 이 **"Vulnerable version"** 으로 배포를 막아요

> 두 요청서 모두 AI 가 만든 코드를 **고치지 않고** Vercel 에 올려 ✅ 저장 → n8n 도착까지 확인했어요. ("최신 버전" 한 줄을 빼면 Next.js 는 실제로 배포가 막혔습니다.)

받은 코드 올리기 — **GitHub → New repository → Add file → Create new file** (파일 이름에 `api/diary.js` 처럼 쓰면 폴더가 생겨요) → 코드 붙여넣기 → **Commit** → Vercel **Import** → AI 가 알려 준 **환경변수 이름 · 값** 넣기 → **Deploy**

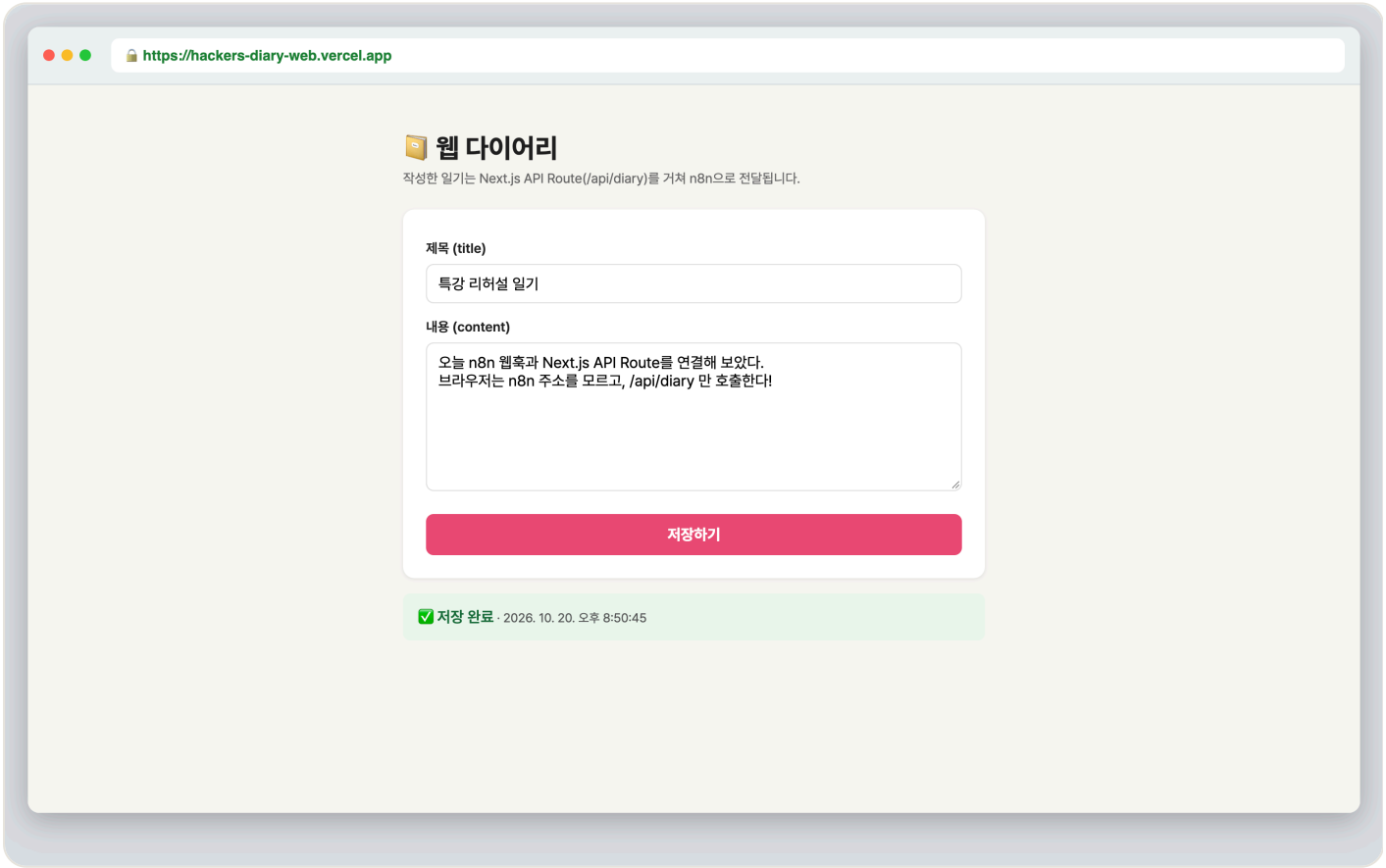
> 🔎 확인 한 가지: 화면 코드(`index.html` · `page`)에 n8n 주소가 **없고** `/api/diary` 같은 내 주소만 있으면 성공. 막히면 에러 메시지를 그대로 붙여 넣고 **"고쳐 줘"**.

> 💬 AI 가 만든 사이트는 성공 문구가 **"저장되었습니다!"** 처럼 다를 수 있어요. 저장 후 n8n **Executions** 에 도착했으면 성공이고, 과제 채점에서도 "저장 완료 **같은** 성공 표시"로 인정됩니다.

자주 막히는 곳

화면 메시지	해결
“N8N_WEBHOOK_URL 환경변수가 없습니다”	Vercel → Settings → Environment Variables 추가 → **Deployments → ⋯ → Redeploy**
“n8n 응답 오류 (HTTP 404)”	**Production URL** 인지, n8n 에서 **Publish** 했는지
저장하면 404 (심플 버전)	`api` 폴더가 **맨 위**에 있는지 (`api/diary.js`)
빌드 실패 “Vulnerable version of Next.js”	`package.json` 의 `next` 를 최신 버전으로 — AI 로 만들었다면 “Next.js 최신 버전으로 고쳐 줘”

✅ 문제 2 정답 화면



문제 2 정답 — 배포된 Vercel 다이어리에서 저장 완료

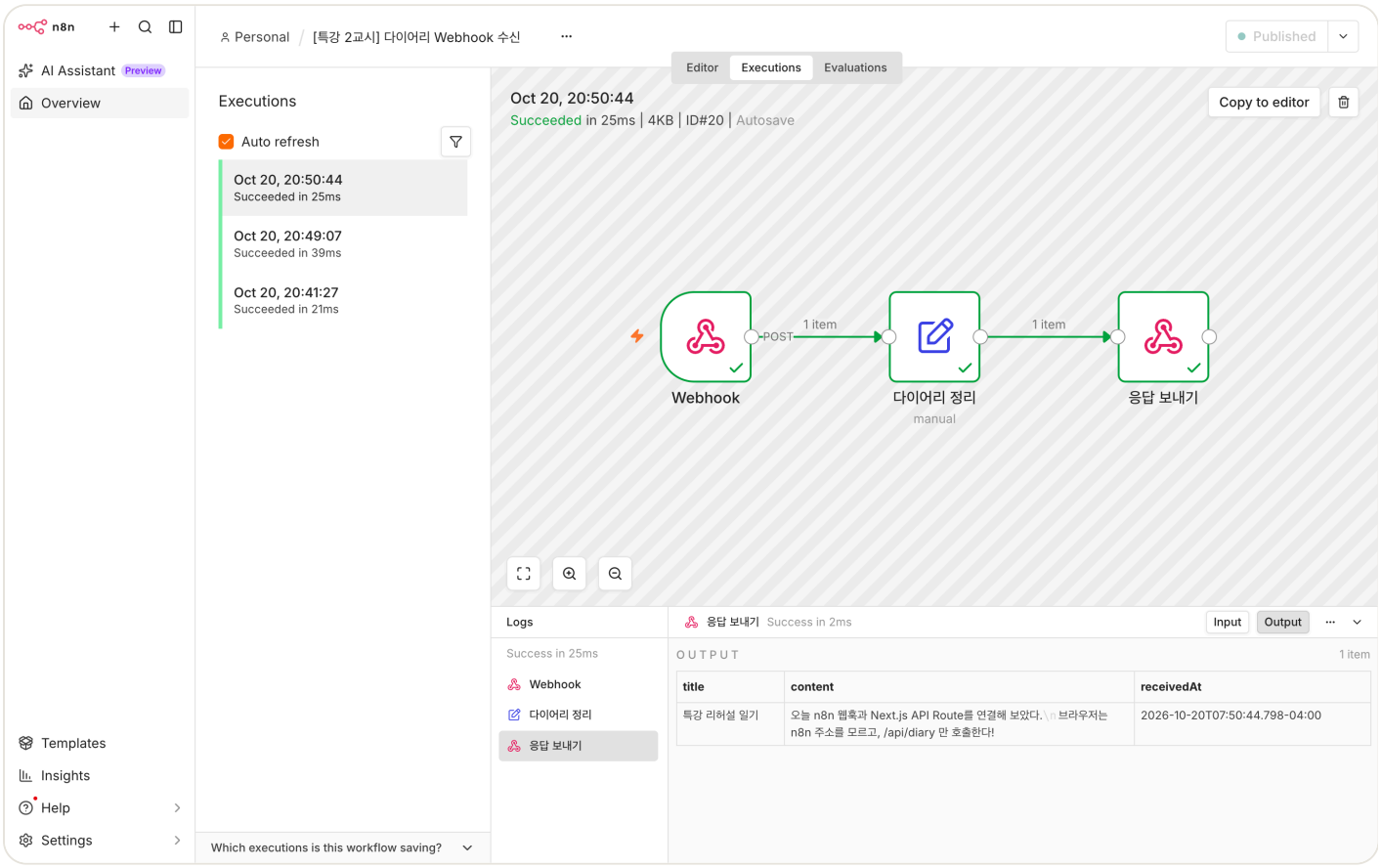
- [ ] 가) 주소창에 **vercel.app 도메인 (10점)**

☐ 나) **title / content** 입력 폼 + 다) **제출(저장) 버튼 (10점)**

☐ 라) **저장 성공 UI** — “저장 완료” 문구 · 토스트 · 결과 영역 등 형태 무관 **(10점)**

☐ 브라우저 → **Next.js API Route** `/api/diary` → n8n 구조 (n8n 직접 호출 ❌) **(10점)**

n8n 쪽에서도 도착을 확인할 수 있습니다 👇



n8n Executions — 다이어리 수신 기록

3교시 · Hacker News RSS 수집과 Limit

📚 이번 교시 필수 개념

⏰ 트리거 종류 · 🗞️ RSS · XML · 📦 아이템(item) · 🚧 Limit · ✏️ Expression {{ }}

목표: **밖에서 정보를 가져와(수집) → 필요한 만큼만 고르고(제어) → 쓸 칸만 남기는(가공) 데이터 파이프라인**

트리거 — 워크플로우의 시작 버튼 ⏰

트리거	비유	언제 쓰나
Manual Trigger (수동 실행)	🖐️ 내가 직접 누르는 **시작 버튼**	만들면서 테스트할 때 ← **오늘 3교시**
Schedule Trigger	⏰ **알람 시계**	매일 08:00, 매주 월요일 …
Webhook	🔔 **초인종**	다른 프로그램이 부를 때 ← **1·2교시**
앱 트리거 (Gmail, 시트 …)	📬 **우편함 알림**	새 메일 · 새 행이 생기면

> 실습은 **수동 실행**으로 확인하고, 완성되면 **Schedule** 로 바꾸기만 하면 매일 자동으로 돕니다.

RSS — 신문 정기구독 🗞️

- 신문사 홈페이지를 돌아다니며 새 기사를 찾는 대신(= 크롤링)

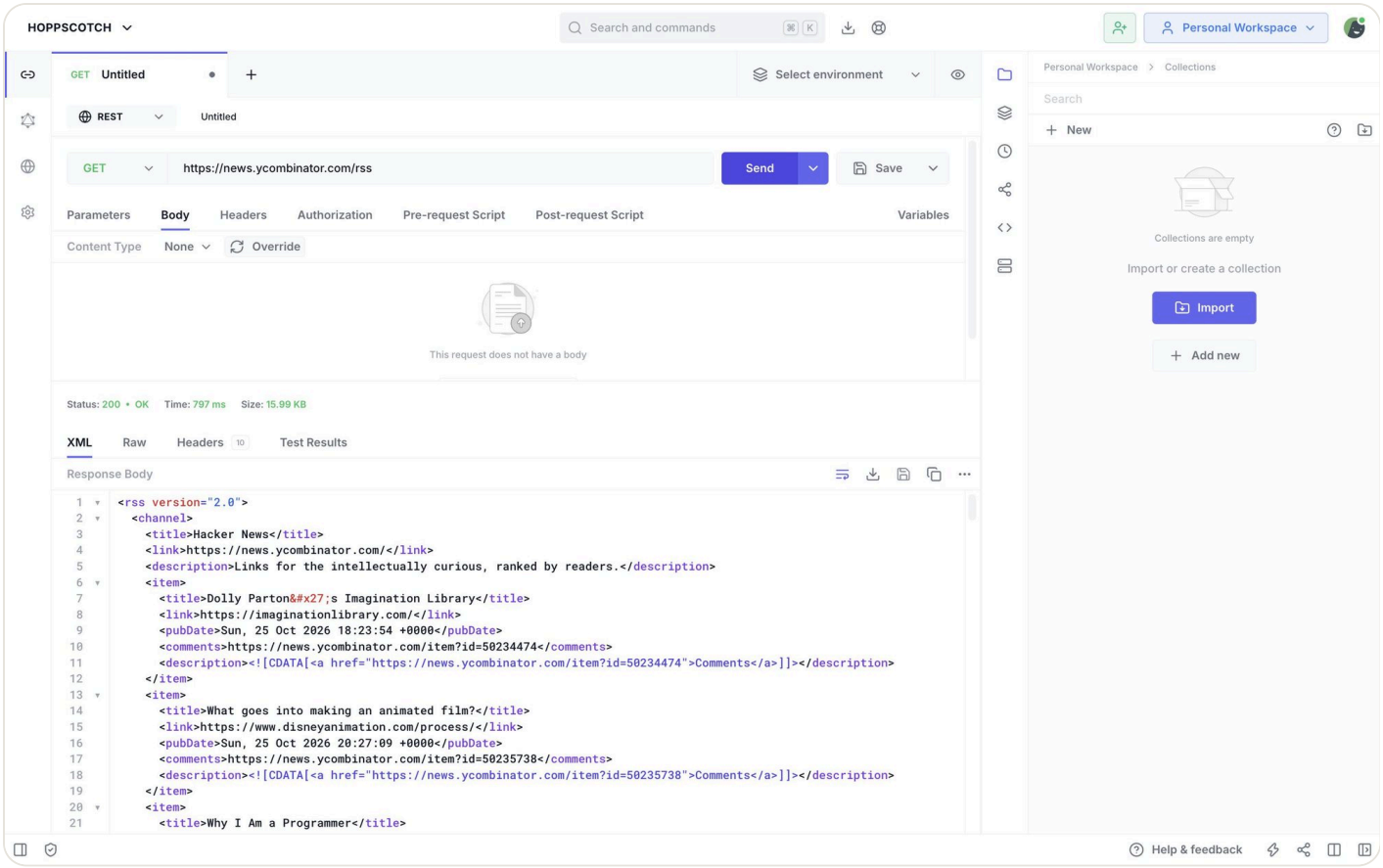
- 신문사가 **“새 기사 목록”을 정해진 양식으로 현관에 배달**해 주는 것 = **RSS**
- 주소 하나면 끝 → `https://news.ycombinator.com/rss`
- 💡 크롤링은 “설계도를 뜯어보며 원하는 방 찾기”, RSS 는 “현관의 신문 받기” → **가능하면 RSS 부터!**

RSS 는 XML, n8n 은 JSON

- RSS 원본은 **XML** 이라는 꺾쇠(`<title>…</title>`) 형식 — 브라우저로 열면 보입니다.
- **RSS Read 노드가 자동으로 JSON(주문서)으로 바꿔** 줍니다 → 우리는 `title`, `link` 같은 이름표만 보면 됩니다.

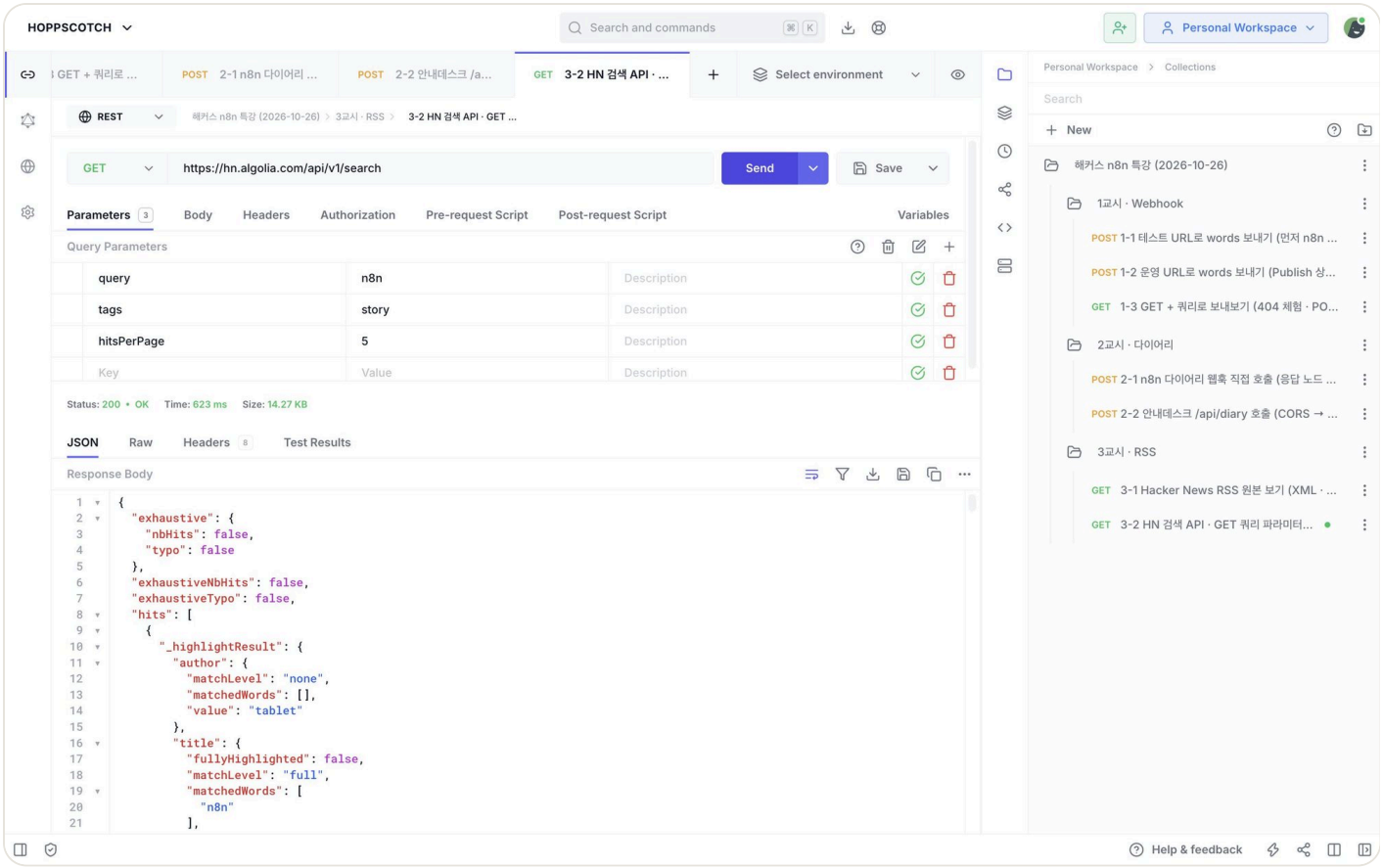
```
XML  :  <item><title>OpenBSD 8.0</title><link>https://…</link></item>
          ↓  RSS Read 노드가 번역
JSON :  { "title": "OpenBSD 8.0", "link": "https://…" }
```

> 🗞️ Hoppscotch 컬렉션 `3-1 Hacker News RSS 원본 보기` 로 신문 원본(XML)을 직접 열어 볼 수 있습니다. (CORS 때문에 **Proxy 모드** 필요 — 1교시 팁 참고)



3-1 Proxy 모드로 본 Hacker News RSS 원본 XML

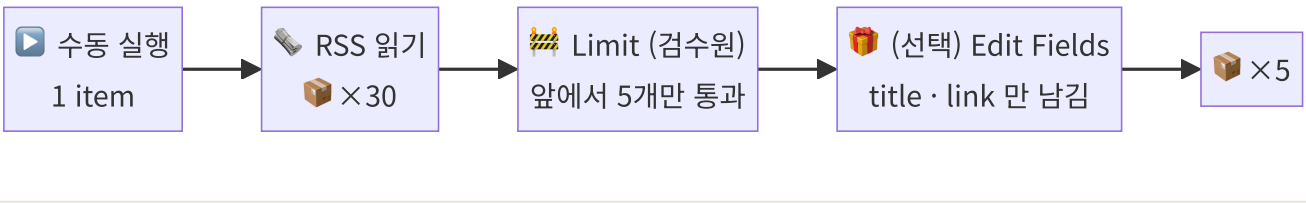
🔍 `3-2 HN 검색 API` 는 **GET + 쿼리 파라미터** 예시입니다. Parameters 표의 `query=n8n`, `tags=story`, `hitsPerPage=5` 가 주소 뒤 `?query=n8n&tags=story&hitsPerPage=5` 로 붙어 나가고, **5개만** 돌려받습니다 (= API 쪽에서 하는 Limit!). 이 API 는 CORS 를 열어 둬서 Browser 모드로도 됩니다.



3-2 Parameters 표로 만든 GET 쿼리 — n8n 관련 기사 5개

아이템(item) — 컨베이어 벨트 📦

- n8n 은 데이터를 **컨베이어 벨트 위의 상자들**로 다룹니다.
- RSS 노드가 기사 30개를 읽으면 → **상자 30개 (30 items)** 가 벨트 위로
- 다음 노드는 **상자 하나하나에 똑같은 작업**을 합니다.



- **Limit** = 벨트 중간의 **검수원** — “앞에서 5개만 통과!”
- (선택) **Edit Fields (Set)** = **재포장** — 필요한 이름표(title, link)만 남기기

Expression `{{ }}` — 빈칸 채우기 ✏️

- 상장 · 안내문 **"___ 님께"** 의 빈칸에 이름을 자동으로 채우는 **메일 머지**와 같습니다.
- n8n 에서 `{{ }}` 안에 쓰면 **"앞 단계에서 온 상자의 값"** 으로 바뀝니다.

이렇게 쓰면	이렇게 바뀜 (상자마다)
`{{ $json.title }}`	`OpenBSD 8.0`
`{{ $json.link }}`	`https://www.openbsd.org/80.html`

- `$json` = **지금 처리 중인 상자(아이템)의 주문서**, `.title` = 그 안의 **이름표**
- 화면 왼쪽 INPUT 에서 필드를 **끌어다 놓으면** 자동으로 써집니다 (직접 타이핑 안 해도 OK)

실습 3 · 뉴스 5개만 골라오기

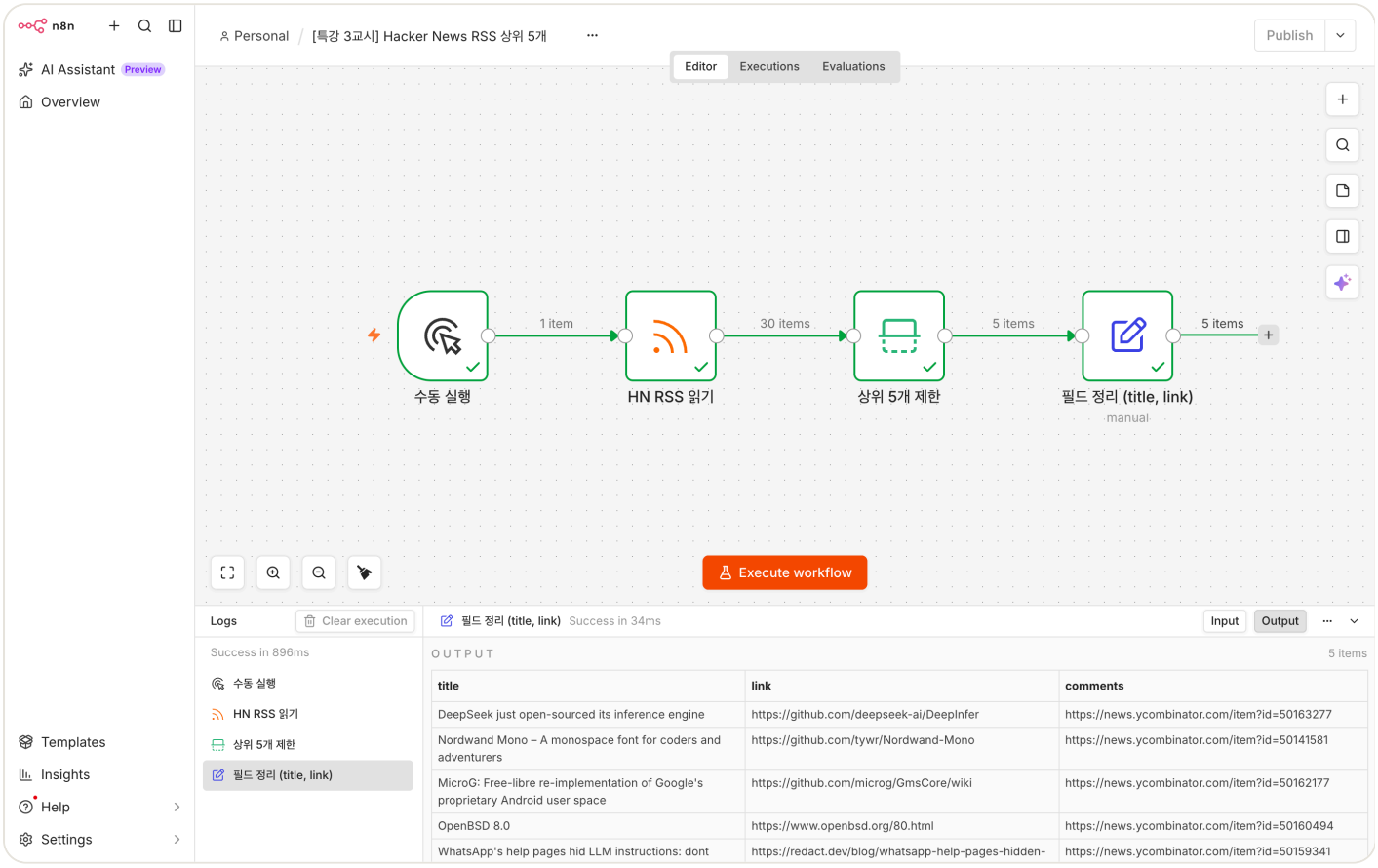
🎬 **따라하기 영상 · 과제 3 뉴스 5개만** — 빈 캔버스에서 노드를 하나씩 만들어 가는 과정 (약 2분 10초) — 영상은 강의자료 웹(hackers-n8n-lecture.vercel.app)에서 재생

📦 **셀프 학습 키트 · 실습 3 워크플로우**

- 03_hn_rss_limit.json — Import from URL: `https://raw.githubusercontent.com/joosung80/hackers-n8n-selfstudy-kit/main/workflows/03_hn_rss_limit.json`
- 📘 매뉴얼 4장 — 빈 캔버스에서 직접 만들어 보기

1. 새 워크플로우 → **Manual Trigger** (수동 실행)
2. `+` → **RSS Read** → URL: `https://news.ycombinator.com/rss`
3. `+` → **Limit** → **Max Items: 5**

4. (선택) `+` → **Edit Fields (Set)**
   - `title` = `{{ $json.title }}`
   - `link` = `{{ $json.link }}`
5. **Execute workflow** → 선 위 숫자 **1 → 30 → 5 items** 확인
6. **Limit** 노드 더블클릭 → 왼쪽 INPUT(30개) · 오른쪽 OUTPUT **Table** 탭 (5줄)
7. 📸 **캡처!**

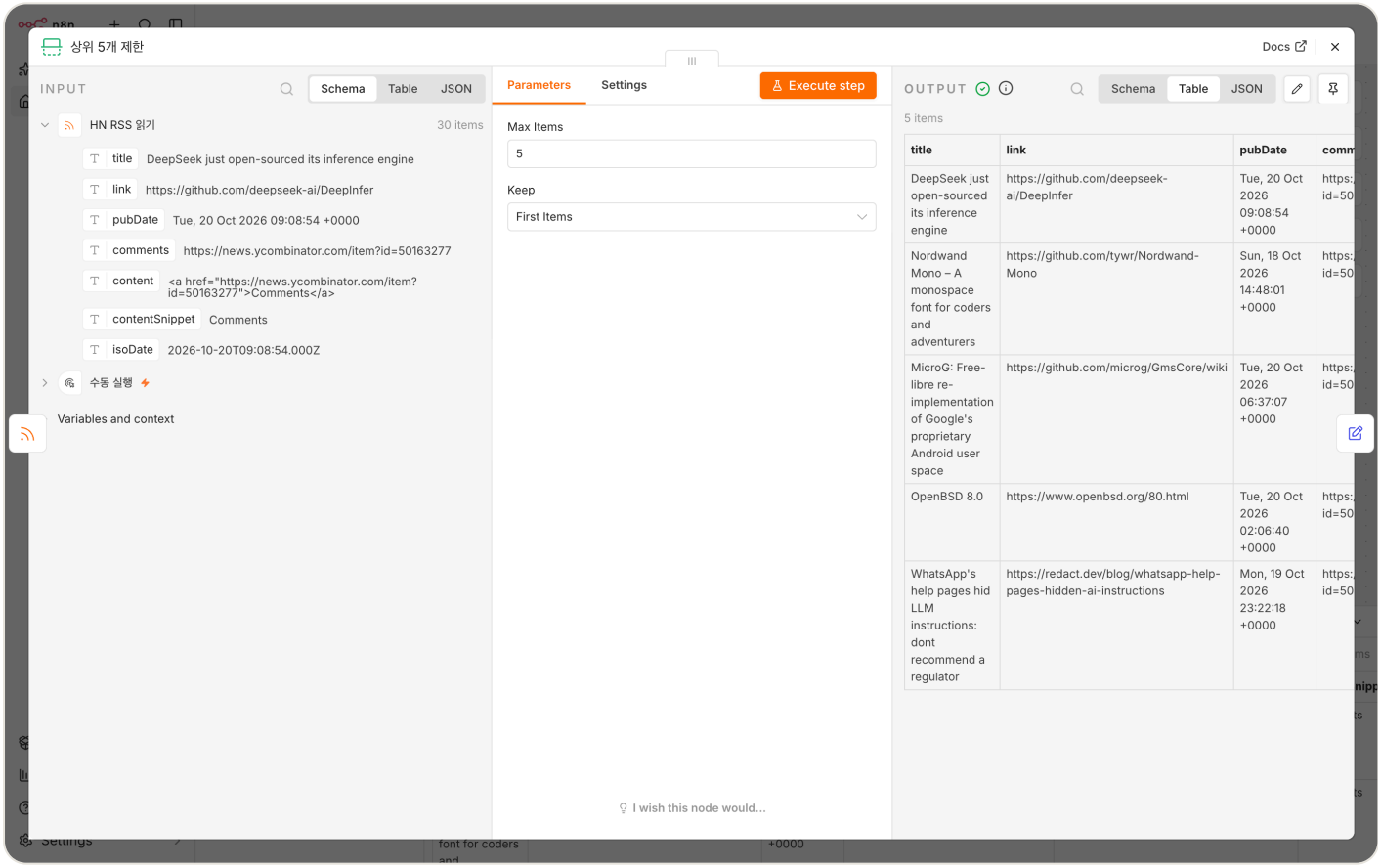


실행 결과 — 30 items 가 Limit 를 지나 5 items 로

자주 막히는 곳

증상	해결
RSS 노드 “Feed not found”	URL 오타, `https://` 누락
Limit 뒤에 1개만 나옴	HTTP Request 로 읽어 **한 상자에 통째로** 담김 → **RSS Read** 노드 사용
5개가 한 화면에 안 보임	OUTPUT 을 **Table** 탭으로, 노드 창 넓히기

✅ 문제 3 정답 화면



문제 3 정답 — Limit 노드 INPUT 30개 / OUTPUT 5개

- [ ] RSS 실행 성공, **INPUT 에 데이터(30 items) (15점)**
- [ ] Limit **OUTPUT 정확히 5 items**, 모두 **title · link** 존재 **(15점)**

마무리

오늘의 그림 다시 보기

단어	비유	오늘 한 일
API	🍽️ 웨이터	n8n 노드들이 일하는 방식
JSON	📝 주문서	`{ "words": [...] }` 보내기
Webhook	🔔 초인종	n8n 이 주문서를 받아 자동 시작
API Route	🛎️ 안내데스크	브라우저 대신 `/api/diary` 가 n8n 호출
환경변수	🔐 금고	`N8N_WEBHOOK_URL` 숨기기
배포	🏭 개업	Vercel 로 전 세계 공개
RSS	🗞️ 신문 구독	Hacker News 최신 글 받기
아이템	📦 컨베이어 상자	30개 → Limit → 5개
워크플로우	🍳 레시피	트리거 → 노드 → 연결선 → 결과
n8n vs AI	🏭 생산 라인 vs 🧠 두뇌	AI 는 생각, n8n 은 순서·연결·반복

과제 제출 📸 × 3

1. **문제 1** — Webhook OUTPUT 에 `words` 3개
2. **문제 2** — Vercel 주소창 + 입력 폼 + 저장 버튼 + 저장 완료
3. **문제 3** — Limit 노드 INPUT + OUTPUT 5개 (title · link)

🎁 선물 — 셀프 학습 키트

집에서 혼자서도 처음부터 끝까지

셀프 실습 매뉴얼 · 설명 메모가 붙은 워크플로우 4개 · REST Client 파일 · 다이어리 웹 소스 · 개념 그림 7장 · 정답 화면

GitHub 에서 보기

⬇️ 전체 zip 다운로드

더 배우고 싶다면

- 온라인 과정 **7~12차시**: n8n 설치 · SNS 콘텐츠 자동화 · 크롤링 · AI Agent · Next.js · 배포
- n8n 공식 문서 — docs.n8n.io
- 다이어리 템플릿 — github.com/joosung80/hackers-n8n-diary-template
- 셀프 학습 키트 — github.com/joosung80/hackers-n8n-selfstudy-kit

감사합니다 🙇